澳大利亚国立大学

计算机学院

JHQ-GPU

面向表示的 GPU 近似最近邻搜索与工作负载级精排预算校准

Rui Luo

导师：Dr Mengxuan Zhang

中文阅读对照稿

章节、公式与图表编号对应英文稿。图内保留英文标签，便于与原始实验及英文稿核对。

COMP8800 — Advanced Computing Research Project

2026 年 10 月

摘要

近似最近邻（ANN）搜索支持大规模嵌入向量集合上的相似性检索。GPU 可以加速搜索，但有限显存使压缩索引具有吸引力。JHQ 是一种层级量化方法：先用紧凑编码筛选候选，再利用残差编码对较小的候选集精排。本文开发 JHQ-GPU，研究如何利用这种表示实现高效的 GPU 建库与搜索。

实现将 GPU 模型拟合和分批编码，与有界输入暂存、打包且按倒排列表排列的布局结合。扫描将每张 256 项距离表替换为两张 16 项表，保持实数算术下的分数。分组扫描在固定的已计算分数与位置排序下，保持精确的全局候选选择。校准通过比较当前结果与较大预算参考返回的近邻，在无需真值标签的情况下决定精排候选数量。

在 NVIDIA RTX 5090 上，跨方法评测使用 Vogue、arXiv、OpenAI-1536 和 OpenAI-3072。在召回率 0.97、batch 1,000 下，JHQ-GPU 在这四个数据集上的主机到主机吞吐为图索引 CAGRA fp32 的 1.32–1.75×，并在其中三个数据集上取得已测方法的最高吞吐。对于更大规模的 BGE 和 Stella 文本嵌入数据集，原始 FP32 向量的存储需求已超出设备显存，JHQ-GPU 仍可在单张 GPU 上完成搜索；最大索引包含 17.78 百万个向量，常驻 GPU 显存为 20.44 GiB。

致谢

感谢导师 Mengxuan Zhang 博士提出研究方向，并在项目过程中通过定期讨论提供指导。

研究与 AI 辅助

我使用 OpenAI Codex 辅助实质性起草、修改、LaTeX、分析及代码 / 日志核查 [30]；使用 ChatGPT 辅助规划与审核 [29]；使用 Anthropic Claude 辅助实现、实验及审核 [4]。选定提示词与使用详情见AI 使用记录。我对本论文中的主张负责。

目录

摘要 i

致谢 i

研究与 AI 辅助 i

符号与缩略语 vi

1 引言 1
- 1.1 背景与研究动机 . . . 1
- 1.2 问题定义与研究范围 . . . 1
- 1.3 技术挑战 . . . 1
- 1.4 研究问题 . . . 2
- 1.5 研究贡献 . . . 2

2 背景与相关工作 3
- 2.1 距离与召回率 . . . 3
- 2.2 IVF、PQ、ADC 与精排 . . . 3
- 2.3 JHQ 表示及其假设 . . . 4
- 2.4 GPU 执行概念 . . . 5
- 2.5 基于图的 ANN 索引 . . . 5
- 2.6 压缩搜索与自适应精排 . . . 5
- 2.7 与已有方法的关系 . . . 6

3 研究设计与测量 7
- 3.1 实验比较与控制变量 . . . 7
- 3.2 数据集、查询、真实近邻与划分 . . . 7
- 3.3 计时与显存测量 . . . 8
- 3.4 基线与前沿构造 . . . 9
- 3.5 实验记录与可复现性 . . . 10

4 GPU 索引设计与构建 12
- 4.1 构建与搜索流程 . . . 12
- 4.2 训练、旋转与码本生成 . . . 12
- 4.3 IVF 分配、ID 与物理内存布局 . . . 14
- 4.4 常驻存储、工作区与索引生命周期 . . . 16
- 4.5 构建成本与已测规模 . . . 17
- 4.6 索引表示与验证 . . . 18

5 面向表示的 GPU 搜索 20
- 5.1 查询流程与排序分数 . . . 20
- 5.2 精确的 Cartesian LUT 分解 . . . 21
- 5.3 打包编码与候选并行内存访问 . . . 22
- 5.4 多 block 扫描中的精确全局主码 Top-c . . . 23
- 5.5 共享完整键阈值与执行调度 . . . 26
- 5.6 Lane 协作残差评分与最终 Top-k . . . 29
- 5.7 IVF 探测列表选择 . . . 30

6 工作负载级预算校准 32
- 6.1 精排预算与搜索成本 . . . 32
- 6.2 输出一致性与预算选择 . . . 32
- 6.3 预算搜索与单调性条件 . . . 34
- 6.4 预算选择基线 . . . 35

6.5 校准质量与复用成本 36
6.6 校准复用与适用范围 37

7 实验评估 **38**
7.1 实验协议 38
7.2 跨系统召回率与吞吐比较 38
7.3 Cartesian LUT、打包载入与 Lane 精排 42
7.4 选择正确性与性能 43
7.5 留出预算质量、吞吐与回本 44
7.6 Batch 大小、调度与工作负载不均衡 48
7.7 索引构建、常驻显存与已测规模 49
7.8 数值检查与召回损失分析 52

8 讨论与未来工作 **54**
8.1 设计启示 54
8.2 替代评分与编码实验 55
8.3 内部与外部有效性的威胁 56
8.4 未来研究方向 57
8.5 部署建议 58

9 结论 **59**
9.1 对四个研究问题的回答 59
9.2 贡献、局限与展望 59

参考文献 **60**

插图目录

2.1 候选与评分级联 4
3.1 主机与设备计时边界 9
4.1 构建与搜索流程 12
4.2 GPU 索引的物理布局 15
4.3 一遍与两遍构建 17
5.1 查询流程 20
5.2 主码 LUT 的 Cartesian 分解 21
5.3 候选并行的打包访问 23
5.4 局部覆盖与截断成对归并 25
5.5 共享键过滤与 chunk 循环 27
5.6 十六 lane 残差精排 29
6.1 工作负载级校准与评估 33
7.1 主比较召回率—吞吐前沿 39
7.2 TF32 与 FP32 精度前沿 40
7.3 小 Batch 竞争区间 41
7.4 Cartesian 查找表分组 42
7.5 打包访问与精排对照 43
7.6 不同样本量下的留出风险 46
7.7 有限池风险预测与观测 47
7.8 校准后复用的直接累计时间 48
7.9 依赖 Batch 的执行区间 49
7.10 记录的构建成本 50
7.11 常驻显存分解 51
7.12 大数据集搜索前沿 52
8.1 Mailman 与面向排序的编码 55

表格目录

1.1 技术贡献与支持证据 2
2.1 相关方法与 JHQ-GPU 设计问题 6
3.1 研究问题与实验对照 7
3.2 数据集与训练配置 8
3.3 主比较主要调参维度 10
3.4 实验名称与记录标签 11
3.5 组件实现 11
4.1 每向量存储模型 16
4.2 各构建阶段的行数组占用模型 18
4.3 原始输入的 FP64 重编码审计 19
4.4 表示约束与审计覆盖 19
5.1 查询阶段的选择约束 20
5.2 Cartesian 查找表的权衡 22
6.1 嵌套与非嵌套候选池 35
6.2 预算对照及其作用 36
7.1 召回率 0.97 时的跨系统比较 38
7.2 匹配配置下的 FP32 敏感性 39
7.3 与 JHQ 一同重测的基线 40
7.4 小 Batch 匹配召回比 41
7.5 128 probes 下的精排收益 43
7.6 严格计算分数选择检查 44
7.7 Probe 选择吞吐比 44
7.8 全池预算校准 45
7.9 抽样预算的留出质量 45
7.10 抽样校准经济性 46
7.11 500 个复用 Batch 的直接成本 47
7.12 集成执行与调度收益 48
7.13 冷构建与常驻显存 50
7.14 主比较分配范围 51
8.1 替代设计与停止决定 56
8.2 有效性威胁 57

符号与缩略语

符号	含义
N, d	数据库向量数与原始维度
M, D_s	主码子空间数与每子空间维度
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	配置 add 行数、输入暂存行数及分配评分缓冲的独立容量（行数）
n_b, S_{stage}	流水缓冲数及每个输入 / 旋转数组的字节预算
Π	方形正交变换；$q' = \Pi q$
B, B_r	每主码子空间位数与每残差坐标位数
$\rho(p), o_\ell$	物理位置 p 的输入 ID，及 IVF 列表 ℓ 的起点
k	返回近邻数
n_c, c	路由候选数及接受精排的主码保留数
α	精排倍率，$c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	样本数；允许缺失的参考槽位数
$C_\alpha, \mathcal{R}_\alpha$	有效保留集合；预算 α 下返回的 ID
$\alpha_{\text{ref}}, B_{\text{reuse}}$	参考精排预算与后续复用 batch 数
g_{LUT}	分解查找表的组数
g_{scan}	每查询扫描组数；零表示单 block 路径
$C_{\text{buf}}, T_{\text{blk}}$	缓冲容量；每 block 线程数
$n_{\text{LUT}}(g_{\text{LUT}})$	每主码子空间 LUT 项数
$\kappa(D, p)$	阈值使用的打包完整键
ℓ_{ref}	每个保留候选的协作 lanes 数
D_{raw}	$\|q - x\|_2^2$
D_{P}	$\|q' - \widehat{p}\|_2^2$
D_{H}	$\|q' - \widehat{p} - \widehat{r}\|_2^2 + \|q'\|_2^2$

缩略语	含义
ANN	近似最近邻
ADC	非对称距离计算
IVF	倒排文件
JHQ	Johnson–Lindenstrauss 增强的层级量化
LUT	查找表
PQ	乘积量化
QPS	每秒查询数
GT	真实近邻
FP32	IEEE binary32；GEMM 模式另行说明
TF32	低精度 Tensor Core 输入；FP32 累加
CV	$\text{sd}(n_c) / \text{mean}(n_c)$

第 1 章引言

1.1 背景与研究动机

向量检索从大型数据库中返回距离查询较近的嵌入向量。GPU 并行计算可以加速距离计算，但原始高维向量可能超出设备显存。压缩可以使更大的索引常驻 GPU，但搜索仍需读取编码并选择有希望的向量。JHQ 是一种层级量化方法，先比较紧凑的主码形成候选短名单，再用残差编码进行更详细的比较 [16]。本文研究如何在遵循这些阶段的评分与选择规则的同时，在 GPU 上高效执行它们，并测量质量与速度之间的权衡。

基于图的 GPU 索引存储向量和图连接；倒排文件（inverted file，IVF）索引则存储在选定数据分区中搜索的向量表示 [18, 28, 33]。这些结构若超出设备显存，完整索引便无法常驻。本文最大的基准数据集 Stella 包含 17.78 百万个 1,024 维文本嵌入向量。其原始 FP32 存储需求、设备可用显存和 JHQ-GPU 常驻分配量，依次说明了容量约束（第7.7节）：

$$S_{\text{raw}} = 67.81\,\text{GiB} > S_{\text{GPU}} \simeq 31.8\,\text{GiB} > S_{\text{resident}} = 20.44\,\text{GiB}.$$

这说明有必要构建常驻 GPU 的压缩索引，直接搜索编码，而非在显存中保存完整的 FP32 数据库。

1.2 问题定义与研究范围

给定向量数据库 $\mathcal{X} \subset \mathbb{R}^d$ 和查询 q，搜索任务是在平方欧氏距离下返回 k 个近似最近邻。本文的研究问题是：如何在单张 GPU 的显存约束内构建并搜索 JHQ 压缩索引，以及如何在给定召回率目标下提高查询吞吐。

搜索先选择相关的 IVF 列表，使用主码粗筛其中的向量，再用残差编码对较小的候选集重新评分。精排预算控制进入最后阶段的候选数量。真值（ground truth，GT）给出原始向量空间中的精确近邻；召回率是其中被找回的比例。

本文保留 JHQ 不改变维度的正交变换和层级编码（第2.3节），研究 GPU 构建、数据布局、搜索执行和精排预算选择。校准不使用 GT，而是比较较小预算与较大预算参考返回的结果，再为固定索引和假设稳定的查询分布复用一个选定倍率。最终召回率通过 GT 评测核查。

评测使用一张 RTX 5090，执行平方 L2 距离的 top-10 搜索。主要跨系统比较使用四个数据集，batch 为 1,000；查找表、内存布局和预算实验覆盖六个数据集。一项小批量研究增加 batch 8/32（第7.2.6节）。索引和核心搜索计算常驻 GPU；计时包含搜索调用内部的查询准备与传输。实现范围不含动态更新、主机分页和多 GPU 执行；十亿规模搜索尚未评测。

1.3 技术挑战

将 JHQ 移植到 GPU 会遇到三个技术挑战。

第一，压缩编码减少了存储，但扫描编码仍可能代价较高。每个八位主码索引一张 256 项的距离查找表（lookup table，LUT）；这些表需要构造，且可能无法放入快速的片上内存。由于 JHQ 通过组合各坐标的量化等级形成编码，这张表可以在实数运算下精确拆成两张 16 项表，分别对应一个字节的前、后四位。挑战在于：更小的表能否抵消额外查表和加法的开销。

第二，小批量可能只有少量查询，不足以充分利用 GPU。将每条查询的候选分给多个线程块可以增加并行工作，但如果各块仅保留相同份额，最好的候选又集中在一个块中，就可能丢失全局优胜者。一种办法是在归并前，让每组各自保留完整的候选短名单，代价是增加

显存和归并开销。挑战在于保留所有可能胜出的候选，同时让这些开销低于并行带来的收益。本文研究共享截断阈值和随批量变化的调度，以降低这些开销。

第三，达到目标召回率所需的精排工作量随工作负载变化。候选过少可能丢失真实近邻，过多则可能花时间读取和计算残差编码，却不改善答案。目标是在不使用真值标签的情况下选择可复用预算。较大预算搜索可以提供参考，但它自身也可能漏掉真实近邻。因此，既要在独立测试查询上核查召回率，也要测量校准和后续搜索的成本。

1.4 研究问题

本文的总问题是：如何通过面向表示结构的执行和工作负载级精排校准，将 JHQ 实现为高效、常驻显存的 GPU ANN 系统。面向表示结构的执行利用编码结构设计数据布局与评分；工作负载级校准从无标签查询中选择一个可复用的精排预算。构建与容量研究提供系统实现基础，跨系统比较则检验其实用价值。由此提出以下四个研究问题。

RQ1：GPU 索引构建与容量。 能否在一张 GPU 上构建并常驻完整的 JHQ 索引？其构建时间、峰值显存和常驻容量分别是多少（第7.7节）?

RQ2：面向表示结构的执行。 如何利用 JHQ 的编码结构设计 GPU 查表、内存访问和候选选择，在遵循各阶段规定的排序与并列规则的同时提高吞吐（第7.3节、第7.4节与第7.6节）?

RQ3：工作负载级精排校准。 能否利用与较大预算参考的输出一致性，在无标签条件下为工作负载选择可复用预算，使独立测试查询上的召回率接近已测预算中的最高召回率，并收回校准成本（第7.5节）?

RQ4：跨系统搜索性能。 按主机到主机边界计时，在什么召回率目标和批量大小下，JHQ-GPU 优于已测试的 GPU ANN 基线（第7.2节）?

第 3 章将这些问题转化为受控比较，分别考察构建了什么、执行是否正确、工作负载需要多少计算，以及系统在哪些区间具有实际价值。

1.5 研究贡献

本文的贡献是 GPU 索引构建流水线、围绕 JHQ 编码设计的搜索执行，以及工作负载级精排校准。表 1.1 将这些新增工作与 JHQ 量化方法及已有 GPU 算法区分开来；后两者属于前人工作。

表 1.1. 技术贡献与支持证据。

	继承基础	项目贡献	证据
C1	JHQ 的变换与层级编码	GPU 模型拟合 / 编码、分批构建及按列表顺序排列的编码	17.78M 向量搜索，常驻 20.44 GiB；拟合与编码为 10.54–10.65 s；采样构建峰值 22.58 GiB
C2	JHQ 编码结构与已有 GPU 算法	双表查找、高效编码读取、使用共享截断阈值的并行选择及协作评分	双表查找在 22/24 配置领先；batch 8 调度吞吐为单 block 的 2.727×；分组选择测试与对相同分数的 CPU 排序一致
C3	候选扩展与输出比较	无标签预算规则、预算搜索条件，以及在独立查询上的质量 / 成本评估	使用完整校准集时，12/12 配置相对最佳已测预算的召回损失最多为 0.0006；四个配置直接测量复用时间

个人贡献。 导师 Mengxuan Zhang 博士提出了将 JHQ 移植到 GPU 的研究方向，并通过定期讨论提供指导。我完成了表 1.1 所概括的工作，包括实验设计、基线比较与分析，并决定采用哪些方案和结论。AI 工具提供了实质性辅助，具体见研究与 AI 辅助说明。

第 2 章背景与相关工作

本章介绍全文使用的搜索目标、压缩表示与 GPU 执行概念，再将 JHQ-GPU 与图索引、量化搜索和自适应精排方法进行对照。

2.1 距离与召回率

对第 1 章介绍的搜索任务，平方欧氏距离为

$$D_{\mathrm{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

评测对每条查询使用十个存档真值（ground truth，GT）近邻 ID。召回率是返回结果命中这十个 ID 的比例，再对查询取平均；这十个 ID 之外的等距候选不计为命中（式 (3.1)）。

2.2 IVF、PQ、ADC 与精排

倒排文件（IVF）索引将数据库划分为多个列表，通常按向量最近的质心进行分配。查询搜索其中 n_{probe} 个列表。若 $\mathcal{L}(q)$ 表示选中列表的编号集合，则可参与评分的向量为

$$U(q) = \bigcup_{\ell \in \mathcal{L}(q)} \mathrm{list}_\ell, \qquad n_c = |U(q)|.$$

位于 $U(q)$ 之外的真实近邻，在向量评分开始之前就已被排除。

乘积量化（product quantisation，PQ）将向量坐标划为 M 个子空间，为每个子空间学习码本，从而压缩向量 [17]。在子空间 m 中，编码 z_m 指向代表向量 $\mathcal{C}_m[z_m]$；这些代表向量共同重建 $\widehat{x}$。非对称距离计算（asymmetric distance computation，ADC）保留未量化的查询，计算

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

其中 q_m 是查询的子向量。每条查询先计算到各码字的距离并制成表，随后扫描数据库编码时使用查表。PQ 分离了不同子空间，但任意学习得到的码本未必能进一步分离同一子空间内部的坐标。JHQ 的主码本具有这一额外结构。

精排对初步评分选出的候选短名单投入更多计算 [10, 18]。令 α 为精排倍率，$c = \max\{k, \lceil \alpha k \rceil\}$ 为请求的短名单大小。在 $|U(q)| \geq k$ 时，以 $C_\alpha(q)$ 表示至多 c 个保留候选，以 $\mathcal{R}_\alpha(q)$ 表示最终返回的 k 个向量，则

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

若 $\mathcal{T}_k$ 为该查询的 GT 集合，由包含关系可得

$$|\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |C_\alpha \cap \mathcal{T}_k| \leq |U \cap \mathcal{T}_k|.$$

增加 probe 数会扩大路由候选池；增加 α 会扩大请求的短名单。要区分路由、粗筛或重构导致的召回损失，需要在保持其他阶段固定的条件下逐项比较。下一节定义 JHQ 的两级评分。

2.3 JHQ 表示及其假设

2.3.1 旋转与层级编码

JHQ 首先对数据库向量和查询应用方形正交变换 $\Pi \in \mathbb{R}^{d\times d}$ [16]。记 $y = \Pi x$、$q' = \Pi q$，则在实数运算下，

$$\begin{aligned}\Pi^\top \Pi = I \Longrightarrow \|\Pi(q-x)\|^2 \\ = (q-x)^\top \Pi^\top \Pi (q-x) = \|q-x\|^2.\end{aligned} \tag{2.4}$$

因此，量化之前的旋转保持维度和距离。有限精度矩阵乘法在第 4 章单独评估。

主码给出 y 的粗略重构 $\widehat{p}(x)$。在评测配置中，每八个坐标组成的子空间使用一个 byte：每个坐标取 $-a$ 或 $+a$ 两档，因此该 byte 记录八个符号位。一般而言，M 个子空间各有 $D_s = d/M$ 维，用 B 位从 $L = 2^{B/D_s}$ 个标量等级的 $K = 2^B$ 种组合中选择。对 L 为整数的支持参数，JHQ 使用 Cartesian 积

$$a_i = \sigma\Phi^{-1}\left(\frac{i-1/2}{L}\right), \quad i = 1,\dots,L, \qquad \mathcal{C}_m = \{a_1,\dots,a_L\}^{D_s}. \tag{2.5}$$

其中 Φ 为标准正态累积分布函数，σ 为根据向量能量估计的尺度。当 $B = D_s = 8$ 时，$L = 2$、$\mathcal{C}_m = \{-a, +a\}^8$，且 $a \simeq 0.67449\sigma$。单位归一化输入使尺度接近 $1/\sqrt{d}$。变换与层级表示均继承自 JHQ。

残差编码用 $\widehat{r}$ 近似剩余误差 $r = y - \widehat{p}$，形成重构 $\widehat{y} = \widehat{p} + \widehat{r}$。评测中的残差量化器学习每个子空间内各坐标共享的标量等级，每个坐标使用一个 byte。因此，每个向量在元数据之外存储 $d/8$ 个主码 byte 和 d 个残差 byte。降低精排预算减少残差读取，已存储的编码保持不变。

该残差相对于主码重构计算。相比之下，使用残差编码的 IVF-PQ 减去向量 x 所属列表的质心 $\mu_{\ell(x)}$：

$$r_{\text{IVF-PQ}} = x - \mu_{\ell(x)}, \qquad r_{\text{JHQ}} = \Pi x - \widehat{p}(x).$$

JHQ 使用 IVF 路由，重构向量时不加回 IVF 质心。

2.3.2 主码与精排评分

主码粗筛比较 q' 与粗略重构：

$$D_{\text{P}} = \|q' - \widehat{p}\|^2.$$

只有短名单中的候选才使用主码和残差码重新评分。实现中的精排分数为

$$D_{\text{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.6}$$

附加项 $\|q'\|^2$ 对同一查询的所有候选都是常数，因此该分数按重构向量的平方距离排序；这一顺序未必与 GT 使用的原始向量顺序相同。分数展开见式 (5.20)。

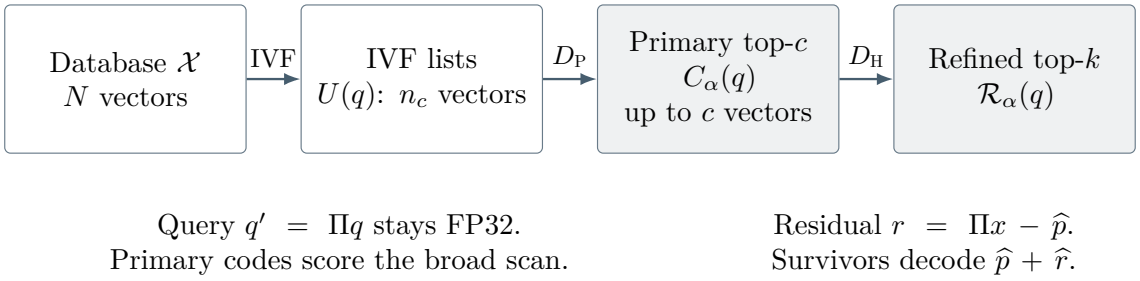


图 2.1. JHQ 的候选与评分级联。IVF 选择列表，主码评分形成短名单，残差码精排该短名单，再执行最终 top-k 选择。

若路由得到 n_c 个向量、主码有 M 个子空间、保留至多 c 个候选，评分工作量为

$$W_{\text{primary}} = O(n_c M) \text{ lookups}, \qquad W_{\text{refine}} = O(cd) \text{ coordinates}.$$

内存访问和候选选择也会增加搜索时间。

2.3.3 建模假设

随机旋转可以分散坐标能量，为上述高斯启发的等级提供动机。距离保持并不意味着坐标独立且服从零均值高斯分布：固定变换将数据库均值 μ 映为 $\Pi\mu$，且 $\|\Pi\mu\| = \|\mu\|$。第 8 章将中心化作为另一种编码选择进行考察。

主码等级取自高斯分位区间的中点，而非使重构均方误差（mean-squared error，MSE）最小的等级。对 $X \sim \mathcal{N}(0, \sigma^2)$ 和零判定阈值，后者为

$$\begin{aligned} a_{\text{MSE}} &= \arg\min_a \mathbb{E}[(X-a)^2 \mid X > 0] \\ &= \mathbb{E}[X \mid X > 0] = \sigma\sqrt{2/\pi} \simeq 0.79788\sigma. \end{aligned}$$

这些数值服务于不同目标。对固定的符号码，任意正的公共主码尺度保持主码排序（式 (4.3)），但残差重构仍取决于尺度。

2.4 GPU 执行概念

GPU 工作组织为线程块；块内线程可通过共享内存协作。warp 是一起调度的 32 个线程。线程块在流式多处理器（streaming multiprocessor，SM）上执行。寄存器、共享内存和线程块数量上限约束并发工作量 [25]。占用率（occupancy）是活跃 warp 数占 SM 可支持 warp 数上限的比例。同一 warp 访问邻近地址时可共用内存事务，称为合并访存（coalesced access）；每块占用较大的表可能减少并发工作量。在这一组织方式下，线程可以分别处理不同查询、为同一查询的不同候选评分，或共同计算一个候选的不同坐标。

候选选择从扫描池中保留最小分数。Faiss 使用 warp 级选择 [18]；其他 GPU top-k 方法将选择工作分布到更大的数组上 [39]。第 5 章解释 JHQ-GPU 使用的选择与调度方法。

2.5 基于图的 ANN 索引

图索引连接相近向量，沿这些边寻找有希望的候选，而不是扫描选中列表中的全部向量。HNSW 使用分层邻近图进行由粗到细的导航 [21]。在 GPU 上，难点是在遍历过程中提供足够的并行工作。SONG 研究并行图搜索 [40]；CAGRA 面向 GPU 设计图构建与搜索 [28]。本文以 CAGRA 作为常驻图索引基线，测试 FP32 和量化向量两种配置。

其他设计通过跨越存储边界扩展容量。DiskANN 将图搜索与 SSD 结合，用于超过主存容量的数据库 [34]；BANG 将主机上的图与压缩 GPU 评分结合 [38]。这些容量增益依赖跨存储边界的通信。

2.6 压缩搜索与自适应精排

优化乘积量化（optimised product quantisation，OPQ）学习旋转和子码本，以减少重构误差 [13]。ScaNN 的各向异性量化则面向内积近似 [15]。这种区别很重要，因为最小化向量重构误差未必最有利于保持近邻排序。

RaBitQ 用随机二进制编码估计距离 [11]，其多位扩展提供更细致的表示 [12]。GPU IVF-RaBitQ 将压缩距离估计、IVF 路由与融合搜索结合 [33]。原始单比特 RaBitQ 与本文评测的 JHQ 主码均存储旋转后的符号。JHQ 使用固定重构等级及残差码精排；IVF-RaBitQ 基线测试每维 4/8/9 位的配置。第7.2节与第7.2.6节 比较完整系统的召回率与吞吐权衡。

Fast Scan 与 Quicker ADC 使用通过单指令多数据（single-instruction, multiple-data, SIMD）操作访问的小表，加速 CPU 距离计算 [1, 2]。因此，表大小与硬件查表支持对速度至关重要。JHQ-GPU 利用主码本内部的坐标结构拆表；第 5 章分析恒等式与有限精度影响。

Faiss 和 GPU 搜索库 cuVS 提供控制精排候选数的参数 [7, 10]。Faiss 自动调参使用带标签的评估 [9]；学习式提前终止预测何时停止 [19]；DARTH 用召回目标表达搜索工作量 [6]。这些方法为第 6 章的无标签校准提供比较依据。

2.7 与已有方法的关系

表 2.1 汇总上述图索引、压缩搜索和自适应计算量方法的设计权衡。对 JHQ，需要研究的是两级表示如何影响 GPU 构建、候选选择与工作负载级精排。第 4–6 章展开 JHQ-GPU 的方法；第 7 章使用第 3 章规定的实测基线与对照进行评估。

每维九位的 IVF-RaBitQ 与 JHQ 每维一位主码加八位残差的名义编码载荷相同。性能前沿也包含每维四位和八位的 IVF-RaBitQ 配置；总显存用量单独评估。

表 2.1. 相关方法及其引出的 JHQ-GPU 设计问题。

方案	主要权衡	JHQ-GPU 的问题
IVF-PQ / OPQ	码本以降低重构误差为训练目标；子空间内部坐标可能耦合	主码的坐标结构如何简化 GPU 评分?
Fast Scan / Quicker ADC	小表在硬件和精度约束下加速查找	拆表能否抵消额外 GPU 查表?
SONG / CAGRA	图导航减少评分，但增加图存储和依赖访问	压缩扫描在哪些区间能与常驻图搜索竞争?
BANG	主机存储扩大容量，但增加通信	单 GPU 常驻完整压缩索引能实现什么?
IVF-RaBitQ	多位编码以存储和 probe 工作量换取距离估计质量	相同召回率下，JHQ 两阶段搜索与 IVF-RaBitQ 相比表现如何?
自适应计算量	质量控制可能需要标签、预测或校准	工作负载一致性能否以可接受的质量和成本选择有效预算?

第 3 章研究设计与测量

本章说明实验如何比较、使用哪些数据集与查询划分、怎样测量时间和显存、如何调优基线，以及哪些记录支持结果的解读与复现。

3.1 实验比较与控制变量

每个测试配置，即*配置单元*（*cell*），固定数据集、索引和工作负载。组件比较改变待研究的机制，并在适用情况下固定训练索引、查询、probe 数、精排预算和评分表示；跨系统比较则使用各系统自己的参数网格（第7.2节）。表 3.1 将研究问题对应到这些对照和测量指标。成对测试衡量执行改变带来的效果，而缓存行为等硬件原因需要单独剖析。

以 Q 表示每秒查询数（QPS），成对加速比为 $Q_{\text{treatment}}/Q_{\text{control}}$，大于一时表示处理方案更快。对 J 个配置使用比值的几何平均，各配置权重相同：

$$G = \exp\left(\frac{1}{J}\sum_{j=1}^{J}\log\frac{Q_{\text{treatment},j}}{Q_{\text{control},j}}\right).$$

表 3.1. 研究问题、受控比较与检查。构建、执行、预算选择和跨系统搜索需要不同的对照。

研究问题	比较	共同条件	指标与检查
RQ1：GPU 索引构建与容量	冷构建与训练模型复用；压缩与原始存储	N, d、表示、路由及构建计时边界	训练 / 编码时间及常驻 / 峰值显存；实测与模型估算的显存对比
RQ2：面向表示结构的执行	查表分组、打包读取、协作精排、分组扫描、共享阈值与 probe 选择	相同索引、负载与预算；共同的评分定义	主机调用 QPS、阶段时间，以及按对应并列规则核对的分数和返回 ID 差异
RQ3：工作负载级精排校准	无标签抽样 / 全池校准；固定 64、100、200；完整预算网格搜索及使用 GT 的诊断选择	相同索引与路由；分开的校准和评估查询	测试查询相对最佳已测预算的召回损失、所选预算 QPS 与校准成本；检验质量与回本
RQ4：跨系统搜索性能	不同 batch 大小下分别调优的 JHQ、CAGRA、IVF-PQ 与 IVF-RaBitQ；暴力搜索参照	相同导出向量、GT、k、GPU 和主机调用边界；显存单独报告	匹配召回率 QPS、显存与覆盖范围；记录胜负

3.2 数据集、查询、真实近邻与划分

表 3.2 列出六个数据集。主要跨系统比较使用前四个；预算、查找表和规模测试覆盖全部六个。Vogue 是服装图像嵌入，arXiv 是摘要嵌入。两组 OpenAI 数据来自 Qdrant 的不同维度 DBpedia 导出 [5, 31, 32, 35]，共享语料来源；对齐检查见第7.2节与第7.2.3节。BGE 使用意大利语 Wikipedia，Stella 使用 TREC24 BioGen 语料 [36, 37]。下表规模对应准备后的导出数据。JHQ 使用 $M = d/8$、$B = B_r = 8$、八次粗量化训练迭代及 100,000 个数据库行进行残差训练。缓存运行复用模型但重新编码数据库；冷运行重新拟合模型。

表 3.2. 数据与 JHQ 训练配置。n_q 为实际查询数，$n_t = 39n_{\text{list}}$ 为请求的粗量化训练样本数。主比较使用前四行。所有配置使用平方 L2 距离及 $k = 10$。

数据集	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI / BGE / Stella 的准备过程删除含 NaN 或无穷值的向量（非有限行），并归一化非零向量：

$$x \leftarrow x/\|x\| \quad (\|x\| > 10^{-12}).$$

准备脚本使用种子 42 无放回抽取查询行，并将这些行从 base 集合中移除。Stella 最终保留 999 个这类语料查询，替代上游的 65 条查询。本实验因此评估留出语料嵌入的向量近邻，而非带相关性标签的用户查询。

各方法使用相同的导出查询和 GT 文件。准备代码通过 Faiss FlatL2 或显存有界的分块扫描，对 FP32 向量执行穷举平方 L2 搜索来生成 GT。对查询集 $\mathcal{Q}$、返回 ID 集合 $\mathcal{R}_k(q)$ 和存档 GT 前缀 $\mathcal{T}_k(q)$，召回率为

$$\text{Recall@}k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

Vogue / arXiv 的 GT 文件包含 100 个近邻，其余为 20 个，只计前十个。前缀之外的等距候选不会自动计为命中，重复返回的 ID 也不增加命中数。

第 6 章从校准查询中选择精排预算，不使用 GT。偶数下标查询构成校准池，奇数下标查询留作测试，下标从零开始：

$$\mathcal{Q}_{\text{cal}} = \{q_i : i \bmod 2 = 0\}, \qquad \mathcal{Q}_{\text{hold}} = \{q_i : i \bmod 2 = 1\},$$

$$(|\mathcal{Q}_{\text{cal}}|, |\mathcal{Q}_{\text{hold}}|) = \begin{cases} (500, 499), & \text{Stella}, \\ (500, 500), & \text{otherwise}. \end{cases}$$

为检验抽样对预算选择的影响，使用种子 12345，在每种样本量 $S \in \{32, 64, 128\}$ 下分别抽取 64 个子集；全池校准（FULL）使用全部 500 条校准查询。选择预算时只比较搜索返回的 ID。GT 用于后续召回率评估，以及另行构造一个使用标签的预算对照（第3.5.2节与第6.2节）。各子集重复使用同一个查询池，因此这些抽样结果描述该池上的选择变化；推广到未来查询所需的假设见第 6 章。

3.3 计时与显存测量

搜索吞吐采用图 3.1 中同步的主机到主机边界，计入查询暂存、传输和预处理，例如 JHQ 的旋转与 CAGRA int8 的查询量化。各方法均通过页锁定（pinned）主机缓冲上传查询；JHQ 将结果返回普通可换页（pageable）内存，基线返回页锁定内存。JHQ / RaBitQ 由 C++ 调用，CAGRA / IVF-PQ 由 Python 调用，句柄和输出缓冲复用。这些接口成本计入测量，对小 batch 和接近持平的比较尤其重要。

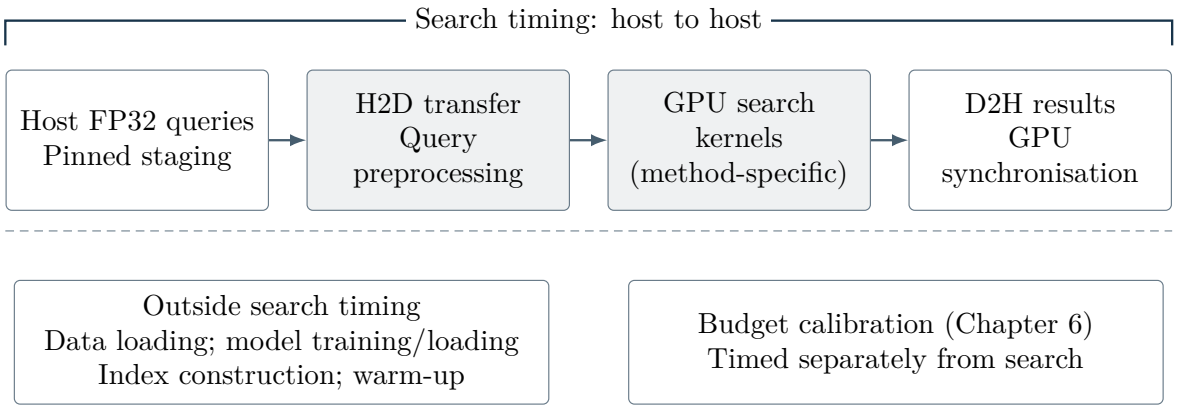


图 3.1. 计时从主机内存中的 FP32 查询开始，到结果返回主机结束，包含输入暂存、主机到设备（host-to-device，H2D）传输、查询预处理、GPU 搜索、设备到主机（device-to-host，D2H）传输及同步。构建、预热和预算校准不计入；构建与校准成本另行报告。GPU 各阶段时间来自单独的剖析运行，其和不一定等于这一端到端时间。

主比较使用大小为 1,000 的 batch，预热后测量五轮，通常每轮 24 次调用。对大小为 b 的 batch，第 r 轮有 m_r 次调用，每次耗时 t_{ri} 秒。先用 batch 大小除以轮内平均调用时间，再取各轮吞吐的中位数：

$$\bar{t}_r = \frac{1}{m_r}\sum_{i=1}^{m_r} t_{ri}, \qquad Q_r = \frac{b}{\bar{t}_r}, \qquad Q = \mathrm{median}_r\, Q_r. \tag{3.2}$$

均值保留有效的慢调用。结果衡量同步 batch 吞吐，而非并发请求的尾延迟。各轮共享同一索引；独立冷构建用于评估训练波动（第7.7节）。预热与运行准备细则见 第3.5.2节。

主比较使用 RTX 5090（32,607 MiB，功率上限 575 W）、驱动 595.71.05、CUDA 13.0 和 cuVS 26.08.01。OpenMP 使用 16 个线程，环境提供 25 个逻辑 CPU。JHQ 内核以 compute_80 PTX 为目标，由驱动在计时前编译为该 GPU 的指令 [27]。GPU 时钟与主机后台活动未受控制，可能带来计时波动。补充实验在后续会话中运行；各项报告说明哪些方法一同计时、是否复用训练模型，以及采用何种数值精度（第3.5.2节、第4.2节与第7.2.3节）。

显存统计区分索引分配、包含上下文和工作区的搜索时进程总占用，以及构建峰值。JHQ 的净值扣除背景分配，CAGRA 的数值则衡量索引分配，二者边界不同（第4.4节与第7.7节）。构建峰值每 20 ms 采样，可能遗漏更短的瞬时峰值；构建时间覆盖 train / add 调用，不含输入载入。

3.4 基线与前沿构造

CAGRA fp32 / int8、IVF-PQ 与 IVF-RaBitQ [17, 28, 33] 使用不同参数网格（表 3.3）；FP32 暴力搜索执行穷举。比较在各自网格内匹配召回率，显存单独报告，并未规定相同存储或调参成本。JHQ 的初始精排倍率在 Vogue / arXiv / OpenAI-1536 / OpenAI-3072 上分别为 64、100、8、4；额外测点将 JHQ 的网格向更高和更低召回率扩展，全部保留在原始记录中。

表 3.3. 主比较的主要调参维度。逐点参数及额外测点保留在原始记录中。网格有限，且不同方法的网格不同。

家族	记录的搜索维度
JHQ	Probes 与 α；初始 probes 为 64/128/256，预算为工作负载预算的 0.5/1/2 倍；额外测点使用最多 1,024 个 probes 及更大 / 更小预算
CAGRA fp32/int8	图度数 32/64；搜索宽度 1/2/4/8；中间 top-k 容量 32–512
IVF-PQ	八位子码；$d = 768$ 时编码 192/384/768 bytes，更高维为 384/768/1,536；fp16/fp32 查表；probes 16–512
IVF-RaBitQ	每维 4/8/9 位、支持的搜索模式、probes 16–1,024；额外 4,096 列表 OpenAI 索引
GPU 暴力搜索	FP32 平方 L2 穷举，共同主机边界

对实测点 a, b，以 R 表示召回率、Q 表示吞吐。前沿保留未被支配的点：若另一点的召回率和吞吐均不低，且至少一项更好，则该点被支配：

$$a \succ b \iff R_a \geq R_b,\ Q_a \geq Q_b,\ (R_a, Q_a) \neq (R_b, Q_b).$$

对目标召回率 r，达标实测吞吐为

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

没有达标测点时，该值未定义。对夹住目标的相邻前沿点，使用对数吞吐插值：

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

插值估计中，若有实测前沿点与目标召回率之差不超过 5×10^{-4}，则按召回率升序使用第一个这样的点，即使它略低于目标。该容差只用于插值估计；上述达标实测吞吐仍要求召回率达到目标。否则，在目标两侧的相邻前沿点之间对吞吐取对数后插值；没有近邻点或两侧测点时，不报告插值估计。

小 batch 比较还允许使用达标的实测端点：若最低实测召回率已超过目标，就使用该点的吞吐，无需外推。当结果接近持平，或两种估计给出的快慢顺序不同时，同时报告两种估计。失败或输出无效的运行不进入前沿。重复调用间返回 ID 顺序变化，本身不会导致测点排除（第3.5.2节）。CAGRA int8 仍纳入比较（图 7.1）。

3.5 实验记录与可复现性

3.5.1 来源与可用记录

项目资料库提供源码、配置、计时和选定输入。完整向量、训练状态及历史可执行文件哈希并非全部可用。严格选择检查中已删除的候选数组需要重新生成，才能独立重新排序。表 3.5 将版本对应到组件对照；各加速比不能相乘得到整系统结果。抽样与泛化限制见第6.2节、第6.3节与第8.3节。

表 3.4列出后文采用的描述性名称；记录标签仅用于查找项目资料库中的原始数据。

表 3.4. 实验名称与记录标签.

正文名称	记录	检查内容	结果位置
主比较	E1	JHQ-GPU 与五类基线；四数据集，batch 1,000	第7.2节
精度检查	D	FP32 与 TF32；沿用存档基线	第7.2.3节
同场次检查	T1	同场次重测 JHQ 与基线	第7.2.4节
严格选择检查	T4	CPU 重排 GPU 计算的候选分数	第7.4.1节
小批量研究	T5	batch 8、32 的跨系统搜索	第7.2.6节
重复冷建库	A1	建库耗时与采样显存峰值	第7.7节
指令检查	A2	共享界读写的编译指令	第5.5节
对齐检查	A3	两个 OpenAI 导出的行对齐	第7.2节
预算决策记录	B	校准选择与全枚举核对	第7.5.5节
表示审计	C1	从原输入重编码及放置检查	第4.6.2节
中心化测试	T2	原始与减均值表示的比较	第8.2.3节

表 3.5. 组件测试使用的实现家族。各实验有自己的对照；版本号不表示连续累积的整系统加速。

版本	机制 / 实验作用
v57	历史 CPU / GPU 比较中的较早 GPU 实现
v61	Word 载入与 lane 精排对照
v62 / v63	分组完整键选择 / 共享阈值
v65 / v69	Batch / 偏斜调度及 radix probe 选择
v69 / v71 / v73	预算校准 / LUT 扫描 / 严格分数导出
v69 / v72	OpenAI：同一训练缓存上的 TF32 与 FP32 add / 查询算术

3.5.2 计时与校准约定

评测配置显式启用十六 lane 精排及自动扫描调度，在单 block 与 2 / 4 / 8 组之间选择，多组时启用共享界；库默认使用标量精排及单 block。可选配额式近似选择器关闭。一次性调度器校准不计入稳态搜索时间，下列三项分别为伪查询收集、首次调用及重复调用中位数的调度器启动耗时：

$$T_{\text{gather}} \in [0.81, 7.04]\,\text{ms}, \quad T_{\text{first}} \in [6.00, 27.77]\,\text{ms}, \quad \text{median}(T_{\text{repeat}}) \in [2.72, 18.13]\,\text{ms}.$$

主比较中，JHQ 每个变体 / 轮次预热一次，基线起初预热三次。驱动即时（just-in-time，JIT）编译和 JHQ CUDA Graph 捕获均在计时前完成；后者记录 GPU 操作，供后续重复执行。慢基线配置依据预热耗时估计，使用较少的计时调用。重复调用间的 ID 顺序变化可能包括并列重排，本身不排除测点；这不同于单次结果内部含有重复 ID。JHQ 没有同样的重复调用诊断标记。

打包载入比较为每个变体保留一项汇总计时。采样预算表同时报告 64 个样本的所选预算中位数与平均质量；平均召回由打印的最佳召回减去已记录平均损失重建，因此两项统计未必对应同一个固定预算。

预算实验中，每个固定预算预热一次、计时三次。校准时间包括参考搜索、试验预算下的搜索，以及在主机上比较这些搜索的结果（$O(Sk^2)$）；不包括收集样本查询、一次性的调度器校准和图捕获。预热遍历全部查询，但其输出不参与预算选择，校准过程也不使用 GT。所选预算 QPS 使用固定预算计时，直接复用测试见 第6.5节。对排序后的 64 个召回损失，报告分位数采用

$$L_{95} = L^{\text{sorted}}_{\lfloor 0.95(64-1) \rfloor} = L^{\text{sorted}}_{59} \qquad \text{(zero-based)}.$$

第 4 章GPU 索引设计与构建

压缩 GPU 索引既需要可搜索的表示，也需要能在设备显存内完成的构建过程。本章说明 JHQ 的训练、编码和按列表组织的存储 [16]，再检查构建后的表示。查询执行与实测构建成本见 第5章与第7.7节。

4.1 构建与搜索流程

索引构建将可复用模型与具体数据库的数组分开。训练阶段拟合模型，add 编码数据库并将各行放入 IVF 列表顺序，搜索随后使用这一完成的 GPU 常驻表示。

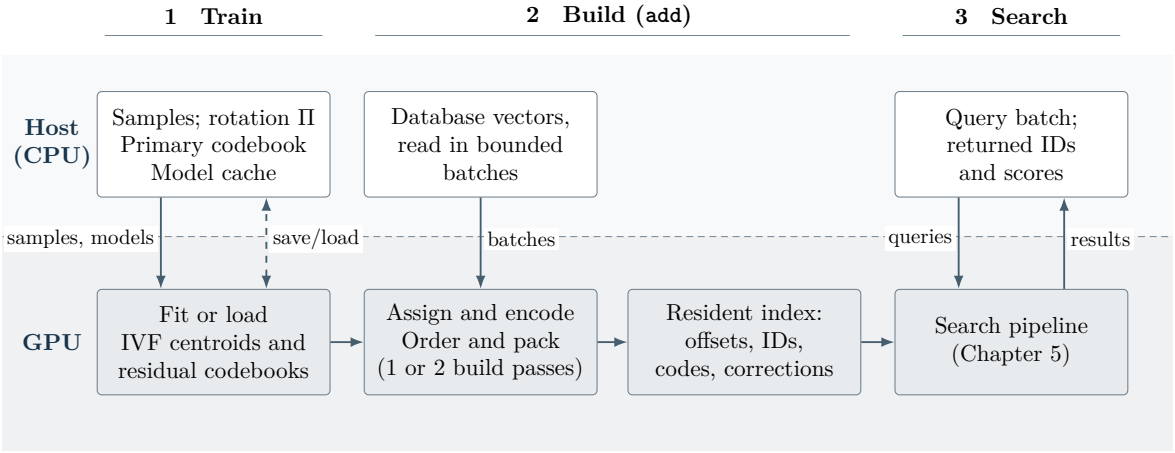


图 4.1. 构建与搜索流程。

主机准备样本、旋转矩阵和解析主码本；GPU 应用旋转，拟合或载入 IVF / 残差模型，并将数据库 batch 编码为常驻索引。可复用模型状态包含旋转、码本和质心。虚线路径表示这一状态的保存与载入：缓存命中会跳过模型拟合，add 仍需编码数据库并按列表排序。

一次成功的 `add` 固定物理位置，后续 add 被拒绝，更新需要重建。返回分数与主机调用计时边界的定义见 式 (5.20) 与第3.3节。

4.2 训练、旋转与码本生成

4.2.1 变换与抽样

主机上的 QR 分解将带种子的高斯矩阵分为正交因子和上三角因子，其中正交因子给出方形 Π。训练样本和数据库向量使用同一变换：

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

维度仍为 d。第4.6节 的验证区分存储矩阵的几何性质（K0）与通用矩阵乘法（general matrix multiplication，GEMM）的舍入。

主评测版本（v69）存储 FP32 矩阵，但在 $d \geq 1{,}536$ 时启用 cuBLAS TF32 tensor-core 运算。这影响两个 OpenAI 数据集在训练 / add / 查询中的旋转和质心 GEMM；实现提供的环境变量开关无法关闭这一模式。因此，FP32 存储并不表示完全采用 FP32 乘法。FP32 变体（v72）修正了这一开关。敏感性实验复用同一个 TF32 训练缓存，改变 add / 查询算术，并非以 FP32 重新训练模型（第7.2.3节）。

主码 / IVF 训练从数据库中等间隔选取样本，请求的样本数为 $n_t = 39n_{\text{list}}$：

$$n_t = \begin{cases} 159{,}744, & \text{Vogue}, \\ 319{,}488, & \text{arXiv/OpenAI}, \\ 1{,}277{,}952, & \text{BGE/Stella}. \end{cases}$$

这是 Faiss 的样本数警告阈值，而非量化质量保证 [8]。

4.2.2 解析主码本

主码本由样本尺度解析生成，其 Cartesian 等级保留了编码器与查找表使用的坐标结构：

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

$$B = D_s = 8 \Longrightarrow \mathcal{C}_m = \{-a, +a\}^8, \quad a = 0.674\sigma.$$

八个符号可放入一个 byte，支持 $O(d)$ 编码与拆分 LUT [16]（第5.2节）。无约束 Lloyd 更新可能破坏这一 Cartesian 结构。对固定符号向量 $s \in \{-1, +1\}^d$，平方距离展开为

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

$$a > 0 \Longrightarrow \arg\min_s \|q' - as\|^2 = \arg\max_s \langle q', s\rangle.$$

因此，在 $a > 0$ 时，主码排序由查询与符号向量的内积决定，无需数据服从高斯分布。残差仍依赖 a；零尺度和 FP32 并列需要另行处理。

4.2.3 粗分配中的筛选与精确计算

IVF 用间隔选取的旋转行初始化质心，并以 $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$ 运行八次迭代。支持的维度先用 int8 分数筛选候选，再在 FP32 中处理不确定情况。下面的引理说明区间筛选保留分配结果所需的条件。

引理 4.1 (具有认证分数区间的分配)**.**
设 $\widetilde{y}, \widetilde{\mu}_\ell$ 为反量化近似，$e = y - \widetilde{y}$，$f_\ell = \mu_\ell - \widetilde{\mu}_\ell$。保持质心范数精确时，近似分数 $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ 的实数算术误差由下式界定：

$$E_\ell = 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big). \tag{4.4}$$

给定包含每个目标分数的有效区间 $[L_\ell, U_\ell]$，仅丢弃 $L_\ell > \min_j U_j$，并对其余分数精确求解，即可保留最小值及规定的并列规则。

证明.
使用 $e = y - \widetilde{y}$ 和 $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$，

$$\langle y, \mu_\ell\rangle - \langle \widetilde{y}, \widetilde{\mu}_\ell\rangle = \langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle,$$
$$|s_\ell - \widetilde{s}_\ell| \le 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big) = E_\ell.$$

对 $j^* \in \arg\min_j U_j$，

$$L_\ell > U_{j^*} \Longrightarrow s_\ell \ge L_\ell > U_{j^*} \ge s_{j^*}.$$

严格不等号排除 ℓ；等号情况仍可入选。 □

有效分数区间是关键条件：若无累加误差界，FP32 安全余量不能证明分配结果逐位一致。存档 FP16 尾部分配比较在 Vogue 的 0/14,838、BGE 的 6/31,748 和 Stella 的 3/16,359 个分配中报告差异。冷构建日志未标明实际采用的分配精度。

4.2.4 残差拟合与迭代预算

残差相对主码重建定义，即 $r = \Pi x - \widehat{p}(x)$。残差拟合使用 100,000 行。在每个子空间内，GPU 排序和 Lloyd 更新拟合 $K_r = 2^{B_r}$ 个有序标量等级，在各坐标间共享，最多更新 2,000 次。

较早的十二单元 Vogue / OpenAI-3072 扫描（$B_r = 4/8$，probe 为 32/128/256）固定主码 / IVF 模型。在 25 次 Lloyd 迭代下，以 100K 行替代全部行最多损失 0.0018 召回率（十一个下降、一个持平）。中间样本量的结果并非单调，因此该证据在这一协议内支持有界残差训练。

历史 100K 训练从 25→2,000 次更新增加了 0.0063–0.0076 召回率。尾部 bin 更新和 BGE / Stella 的结果仍在变化：达到上限并不证明收敛，也不违背精确 Lloyd 的下降性质。20,000 次更新时，历史 Stella 全量 / 100K 召回率为 0.9921/0.9920。

4.2.5 训练工作量与列表不均衡

训练工作量模型分别考虑变换生成、样本旋转、主码本生成和残差提取。以 n_r 表示残差训练行数，变换部分为

$$W_{\mathrm{QR}} = O(d^3), \qquad W_{\mathrm{rotate}} = O(n_t d^2).$$

直接粗分配每次迭代成本为 $O(n_t n_{\mathrm{list}} d)$，其余部分为

$$W_{\mathrm{primary}} = O(MKD_s), \qquad W_{\text{residual extraction}} = O(n_r d),$$

随后进行排序与标量更新。这些量表示操作数。同步诊断表明，BGE / Stella 训练约 83% 的时间在 IVF k-means（约 2.97 s）。该诊断使用一个 add 缓冲并强制同步，不同于 第7.7节 中正常的重叠测量。列表长度不均影响后续扫描工作量，对应 第5.5节；八次 IVF 迭代、间隔种子和有限抽样各自对不均衡的影响未单独测量。

4.3 IVF 分配、ID 与物理内存布局

4.3.1 从数据库 ID 到列表顺序的物理位置

列表排序使分配给同一 IVF 质心的向量连续存储。输入行的标识为 $i \in \{0, \ldots, N-1\}$；粗分配使用计算得到的质心分数，其数学目标为

$$a(i) = \operatorname*{arg\,min}_{0 \le \ell < n_{\mathrm{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

实现采用上文说明的数值分配路径。主码和残差编码不减去 $\mu_{a(i)}$；列表归属独立于这些重建，限制后续搜索空间。

构建器将分配结果与输入 ID 一同排序。令 $\rho(p)$ 为排序后物理位置 p 上的输入 ID，o_ℓ 为列表 ℓ 的起点。完成的数组满足

$$0 = o_0 \le o_1 \le \cdots \le o_{n_{\mathrm{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

lower-bound 搜索找到各列表的首个位置，因此相同偏移表示空列表：

$$o_\ell = o_{\ell+1} \Longleftrightarrow \text{list } \ell \text{ is empty}.$$

位置在完成的索引中保持固定，重建后可能改变。所有数组使用同一个 $\rho(p)$，ID 通过 $p \mapsto \rho(p)$ 映射回输入行。两遍 add 暂时构造 ρ^{-1}，将新编码直接写入目标位置，称为 scatter。主码并列使用物理位置，最终并列使用数据库 ID。

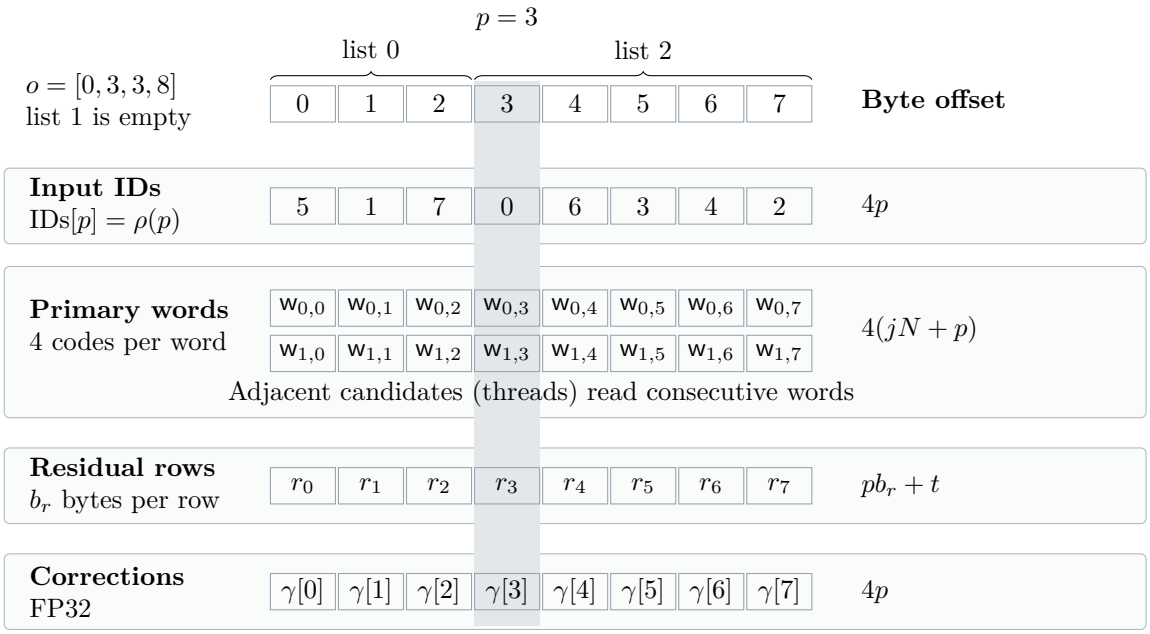


图 4.2. GPU 索引的物理布局.

图中示例取 $N = 8$，offsets $o = [0, 3, 3, 8]$ 使 list 1 为空。浅色列标出 $p = 3$：四个独立数组均以相同物理位置索引，右侧 byte 偏移相对各自数组的起点。固定 word group j 时，相邻候选对应连续的 32 位 word，使扫描相邻候选的线程可以合并访存；八位残差配置下，每行存放同一向量的 b_r 个编码 bytes。

4.3.2 精确地址与表示宽度

主码表示按子空间组织候选。对主码 byte $Z[m, p]$，地址为

$$\mathrm{offset}_{\mathrm{byte}}(Z[m, p]) = mN + p.$$

最终打包将同一向量的四个子空间 byte 合为一个 word：

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \mathrm{addr}_{\mathrm{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

这里 $j = 0, \ldots, M/4 - 1$。word-layout build $\Longrightarrow M \bmod 4 = 0$；其余 M 被拒绝，独立的 byte-layout 编译则保留 byte 表示。每个数组占 NM bytes，重打包暂时同时持有两份副本，成本为 $2NM$。一遍构建将已有编码重排（gather）到 vector-major 的 $[N, M]$ 数组，转置并重打包，逐次释放旧副本；两遍构建直接 scatter 到 subspace-major 存储，再进行相同重打包。搜索时的 lane 映射见 第5.3节。

残差码保持 row-major，每向量占 $b_r = \lceil dB_r/8 \rceil$ bytes。八位配置中，行 p 的坐标 t 位于 byte 偏移 $pb_r + t$；四位路径中，它位于 byte $pb_r + \lfloor t/2 \rfloor$，偶数坐标在低四位（low nibble），奇数坐标在高四位（high nibble）。每个物理行有四个 byte 的校正项，存储

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

gather 或 scatter 时，校正项必须与该行编码一同移动。按列表扫描随后只为保留候选读取残差。精排使用压缩重建；原始 FP32 重排序还需增加 $4Nd$ 个设备 bytes。第 7 章评估其召回率。

4.4 常驻存储、工作区与索引生命周期

4.4.1 存储字节与分配字节

常驻行数据只是设备分配的一部分，模型、查询工作区和临时副本另计。主码以 byte 存储时，行与列表偏移的存储为

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

ID 和校正项贡献 $8N$ bytes。当 $M = d/8$、$B_r = 8$ 时，仅编码载荷为

$$b_{\text{code}} = d/8 + d = 9d/8 = 1.125d.$$

这比 8 位标量量化（SQ8）多 12.5%，因此图存储和向量编码字节数需要一起比较。

表 4.1. 每向量字节数。JHQ 包含编码、ID 和校正项；SQ8 仅含向量编码；CAGRA int8 模型包含向量编码及 32 个四字节邻居。不含 offsets、模型参数、工作区及其他运行时分配。这些组件不是完整进程总量。

d	主码	残差	ID + 校正	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

图度数为 $d_g = 32$ 时，CAGRA int8 每行使用 $d + 4d_g$ bytes。Stella 的 JHQ 行载荷为 19.20 GiB，接近 int8 CAGRA 报告的 19,656 MiB 索引分配，但统计边界不同。改变 α 影响残差读取量，不改变已存储的行字节数。

基本模型存储计入旋转矩阵、主码与残差码本，以及质心和其范数：

$$S_{\text{model}} = 4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1) \quad \text{bytes}.$$

派生数组另计。查询工作区包含两个查询矩阵、质心分数、LUT、probes 和保留候选。对预留查询容量 b_{cap}，可选残差 LUT 增加 $4b_{\text{cap}}dK_r$ bytes，默认直接评分。报告的 128-probe 配置为扫描分组预留约 93.8 MiB，即使未使用也保留该分配（式 (5.13)）；这一大小取决于已分配容量。

输入归调用方所有。对 n_b 个流水缓冲、每缓冲暂存 a_{in} 行，页锁定主机暂存与设备排序数组的存储为

$$S_{\text{pinned}} = 4n_b a_{\text{in}} d, \qquad S_{\text{order}} = S_{\text{inverse}} = 4N.$$

逆映射是临时数组。分配临时区在 gather 前释放（式 (4.10)与(4.13)），因此主机暂存、常驻数据和构建峰值具有不同生命周期。

4.4.2 缓存复用与失效处理

缓存键不含代码版本和全量数据哈希，因此代码或数据变化后需使缓存失效。参数扫描复用模型，冷构建重新拟合；仅凭召回率相近不能证明模型相同。缓存命中仍需 QR、预热、模型上传和 add。Stella 的 6.67 s add 包含初始工作区，不含扫描校准与搜索预热，衡量的是编码调用，而非完整重启时间。

改变 α 无需重训。CUDA Graph 重放会复用查询缓冲，工作区在捕获前扩容。固定返回数量时，改变 batch 大小、保留数、probes、lanes、groups 或 bounds 会触发重新捕获。校准可能绕过捕获，索引拥有的缓冲在释放前保留分配。

4.5 构建成本与已测规模

4.5.1 有界输入 batch 与两遍构建

若将全库原始和旋转数组都放在 GPU 上，需要 $4Nd + 4Nd = 8Nd$ bytes。有界暂存限制这些数组，而压缩索引仍需常驻。对 n_b 个流水缓冲、配置的 add 行数 a_{add} 和每数组预算 S_{stage}，暂存行数及原始 / 旋转设备存储为

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

$$4d \le S_{\text{stage}}, \quad n_b = 3,\ S_{\text{stage}} = 128\,\text{MiB} \Longrightarrow S_{\text{float}} \le 2n_b S_{\text{stage}} = 768\,\text{MiB}.$$

上界只覆盖这些浮点数组，分配和编码器临时区另计。一遍构建按输入顺序编码，再 gather 为列表顺序；两遍构建先分配和排序，再重读并直接编码到最终位置，以额外遍历换取较少的压缩副本。

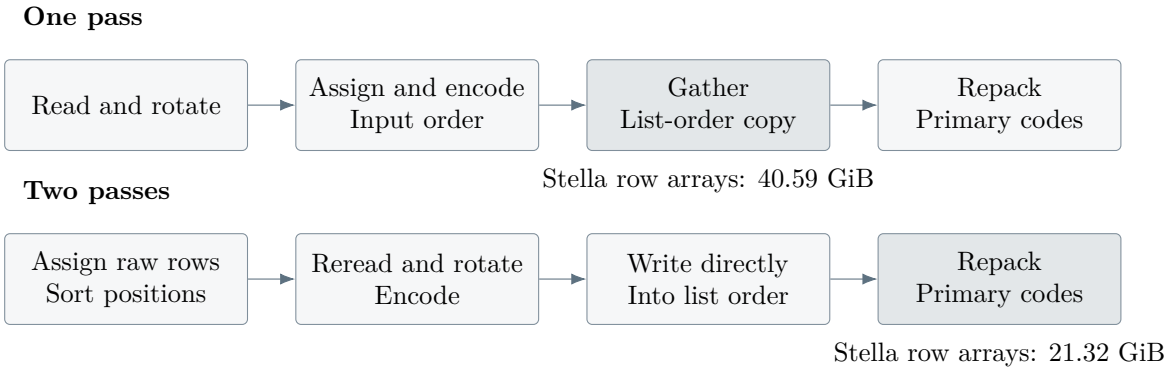


图 4.3. 一遍与两遍构建.

对 Stella，解析模型给出的行数组占用为：一遍 gather 需要 40.59 GiB，两遍 repack 需要 21.32 GiB（表 4.2）。一遍路径同时保留输入顺序与列表顺序的编码；两遍路径直接写入列表顺序。在首遍分配中，质心通过 $\Pi^{\top}$ 变回原坐标系，因而无需先旋转数据库行。

4.5.2 构建各阶段的显存占用

实现根据行数组及其他分配的预留量，估计一遍构建所需显存：

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

十个存活数组贡献

$$3NM + 2Nb_r + 20N = N(3M + 2b_r + 20).$$

1 GiB 预留量是启发式。两遍构建的最终重打包阶段计入 ID、残差、校正项和两份主码，所需存储为

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

整个构建峰值还包括模型、排序、上下文、更早的流水 / 逆置换临时区和后续查询工作区。表 4.2 的阶段值单独计算行数组，并非整个进程峰值。

表 4.2. 行数组占用模型（GiB），$d = 1{,}024$、$M = 128$、$B_r = 8$。BGE 使用一遍，Stella 使用两遍。Stella 的一遍列是反事实容量分析。阶段值不含流水、模型、offsets 和运行时成本。

数据集	最终行载荷	一遍 Gather	两遍重打包
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

分配乘积缓冲使用独立容量 a_{assign}，由 `JHQ_ASSIGN_BATCH` 或源码默认值选择：至少 8,192 个列表时为 32,768，否则为 8,192。其存储为

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

分配容量独立于当前活跃 / 暂存行数。在 $d = 1536/3072$ 下，两个默认值均分配 3 GiB。BGE / Stella 的 $3 \times 4 = 12$ GiB 分配临时区在 gather 前释放。对 BGE，12 GiB 临时区与 23.05 GiB 行数组 gather 不会同时占用显存。峰值是同一时刻存活分配之和的最大值：

$$S_{\text{peak}} = \max_{t} \sum_{A \text{ live at } t} |A|.$$

Stella 冷构建日志选择两遍：式 (4.11) 的一遍估计为 41.6 GiB，同一日志报告尾部阶段可用量为 30.6 GiB。单独采样的常驻进程总量包含上下文和查询工作区：BGE 为 11.93 GiB，Stella 为 20.44 GiB（第4.4节）。补充的 20 ms 设备采样在 BGE add 开头附近观察到 26.08 GiB 峰值，在 Stella 重打包时观察到 22.58 GiB 峰值。较小的两遍重打包量与 Stella 较低的实测峰值一致。这些采样设备总量包含上下文，可能遗漏更短暂的峰值。

4.6 索引表示与验证

表示需要独立于搜索是否成功来验证。此前有符号 32 位的 $n_t d$ 在 2^{31} 溢出，使码本尺度归零，不检查版本的缓存又载入了该值。分块 FP64 范数累加修复了错误，而仅观察路由成功未能发现它。

4.6.1 表示一致性检查

试验导出覆盖 Vogue、arXiv 和 OpenAI-3072。导出器与评测实现共用编码、训练和构建代码；源码比较覆盖该共用路径，而训练模型是否相同以及大数据集两遍构建是否正确，需要另行检查。因此，第一组检查针对导出的表示，而非从原始输入向量重新编码。

码本通过可分性、标量等级和排序检查。FP64 循环检查每个导出行的 $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$，最大绝对误差为 1.28×10^{-7}–2.77×10^{-7}。抽样 GPU 返回 top-10 分数的最大相对误差为 2.38×10^{-7} 至 4.85×10^{-5}，使用包含查询范数常数的 $|s_{\text{GPU}} - s_{64}|/s_{64}$。这些 top-10 返回分数检查跳过不在路由映射中的 ID，因此没有完整检查输出集合成员资格。

试验还将提出的残差距离界与 FP64 重建距离比较，覆盖七个工作负载、896 个查询实例和 72,800,590 个路由候选实例，报告违规数为零。同一批候选也支持 FP64 复合分数恒等式检查；这些表示检查与下文的选择器测试分开。

4.6.2 输入编码与数据放置检查

补充输入审计扩展到 Vogue、arXiv、OpenAI-3072 和 Stella。offsets 单调，端点为 0 和 N，各数据集的 ID 均构成所有数据库行的排列。对存储旋转做 FP64 乘积，$\|\Pi\Pi^{\mathsf{T}} - I\|$ 的最大逐元素误差为 3.96–4.34×10^{-7}。抽样范数检查使用 FP64 旋转，而非 GPU GEMM，测量的是存储矩阵的几何性质。表 4.3 汇总从原始输入重编码的结果。

表 4.3. 原始输入的 FP64 重编码审计。Flips 计数坐标决策；末列为不一致项到残差决策边界的最大距离。结构检查覆盖全部行，重编码使用抽样。

数据集	行数	正交误差	主码翻转	残差翻转	最大边界距离
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

在 768/1,024 维下，重编码差异距决策边界不超过 1.12×10^{-7}。OpenAI-3072 有 2.68% 的残差不一致，距边界最高 2.67×10^{-5}，与 TF32 的影响相符。残差重编码使用存储的主码，避免将主码翻转再次计为残差错误。

按原始 ID 比较一遍与两遍构建，三个较小数据库的编码相同，校正项差不超过 8.94×10^{-8}，Vogue / arXiv / OpenAI 的分配差异为 1/1/40。Stella 只检查两遍构建。未导出质心，因此列表成员归属仍未验证。

4.6.3 验证覆盖范围与局限

表 4.4. 试验和补充输入审计后的表示约束。检查分别处理几何、算术和行放置。

边界	必要性质	证据 / 剩余检查
K0: 几何	共同变换及有界浮点等距误差	四个数据集检查存储矩阵正交性；区分 TF32 GPU 算术
K1: 码本 / 校正	Cartesian 等级、有序残差码本、校正符合解码编码	三个数据库完成试验检查；全行校正误差见第4.6节
K1: 输入编码	存储码对应预期原始输入行	四个样本的 FP64 原始输入重编码；量化边界和 TF32 差异
K1: 放置	Offsets 覆盖各行；ID 构成排列；所有字段随之移动	全行 offsets / ID 检查；三个按 ID 连接的路径比较；质心归属未验证
执行	有效索引、同步和分数顺序约束	存档查询和 Sanitizer 检查；覆盖有限，详见第7.4节

评测使用的打包实现要求

$$M \mid d, \quad D_s \mid B, \quad M \bmod 4 = 0.$$

输入必须为有限值，计数必须有效，未使用槽位需填入标记缺失条目的哨兵值。ID 为有符号 32 位，地址乘积使用更宽算术。严格选择检查验证候选选择（第7.4.1节）；质心归属、空列表行为和对抗尾部仍未认证。

第 5 章面向表示的 GPU 搜索

本章回答 RQ2：如何利用 JHQ 的编码结构组织 GPU 搜索。查表、访存和并行选择利用这一层级表示 [16]，同时保持规定的计算分数排序。第7章 测量这些机制，第6章 再选择各负载的精排预算。

5.1 查询流程与排序分数

查询先选择 IVF 列表，再以主码筛选候选，最后精排保留项并返回 k 个 ID 和分数。这三个决策使用不同分数；GT 则提供独立于搜索过程的第四种排序，用于评估：

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

其中 μ_i 是变换后的质心，$\widehat{p}, \widehat{r}$ 是解码后的主码与残差重建。精排分数在重建平方距离上增加一个仅依赖查询的常数，因此对同一查询保持该距离的排序（第5.6节）。原始向量召回率在第 7 章另行评估。

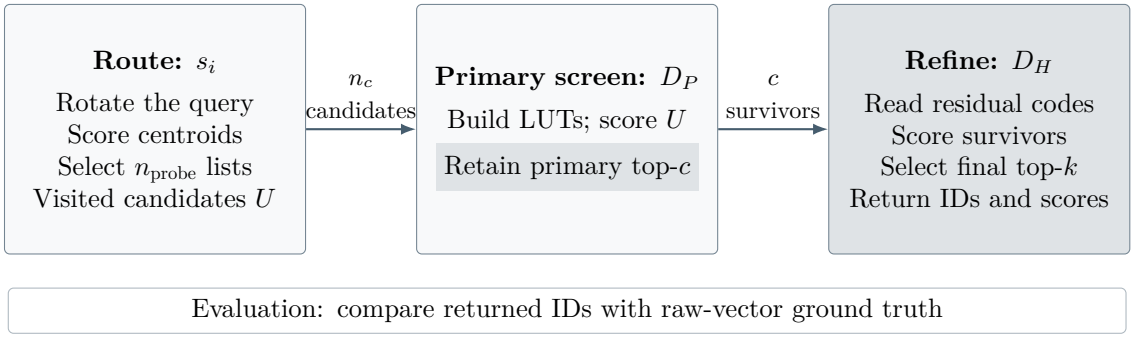


图 5.1. 查询流程.

路由定义访问空间 U，包含 n_c 个候选出现实例。主码筛选最多保留 $c = \max\{k, \lceil \alpha k \rceil\}$ 项供残差评分。图 5.1 的阴影标出依赖该容量的工作：候选保留、残差评分与最终选择。在查询、索引及 probe 数固定时，路由集合和主码评分数量保持不变。下表对照各阶段读取的数据、工作与排序规则。

表 5.1. 查询阶段约束。$c_v = \min(c, n_c)$ 计数有效主码保留候选。精确性针对所说明的计算分数；路由与量化重建决定原始空间近似。

阶段	输入 → 输出	读取数据	主要工作	顺序 / 精确性
变换	$q \to q'$	查询；变换	应用 Π	变换精度见 第7.2.3节
Probe	$n_{\mathrm{list}} \to n_{\mathrm{probe}}$ 个列表	FP32 质心；列表长度	评分；radix；排名计数	(s_i, i)；精确 probe 选择
LUT	$q' \to$ 表	主码本	$MD_s 2^{8/g_{\mathrm{LUT}}}$ 项	实数算术分解；FP32 舍入
主码，$g_{\mathrm{scan}} = 0$	$n_c \to c_v$	打包主码；LUT	$O(n_c M)$ 评分；压缩	按距离精确选择；截断并列时保留候选可不同
主码，$g_{\mathrm{scan}} \geq 1$	$n_c \to c_v$	相同主码表示	评分；压缩；归并	(D_{P}, p)；指定前提下精确
精排	$c_v \to c_v$ 个分数	残差码；校正	$O(c_v d)$	计算的 D_{H}；FP32 舍入可随 lane 分组改变
最终	$c_v \to k$	精排分数；ID	$O(kc)$ 槽位扫描	$(D_{\mathrm{H}}, \mathrm{ID})$；有 k 个有限有效项时精确

评测搜索采用十六 lane 精排与自动扫描调度；它们不同于库的默认设置，配置见第3.5.2节与表 3.5。后续各节依次说明查表、打包访问、分组选择、共享界、精排及 probe 选择（第5.2至5.7节）。

5.2 精确的 Cartesian LUT 分解

扫描需要为每个候选计算主码距离，因而反复查找与编码有关的距离贡献。大表会占用片上存储，或需要从全局显存读取。JHQ 的坐标结构允许 GPU 使用较小的表，而不改变量化器。主码 ADC 使用 FP32 查询 q' 与存储的重建 [16, 17]：

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\mathrm{P}}(q,x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

当 $K = 256$、$D_s = 8$ 时，解析码本包含同一重建幅度下的全部符号组合：

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\mathrm{P}}(q,x) &= \|q'\|_2^2 + da^2 - 2a\sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

其中 Φ 为标准正态累积分布函数；每个 byte 存储八个符号，高位在前。因此，主码排序等价于查询与符号码的内积排序：

$$a > 0 \quad \Longrightarrow \quad \arg\min_x D_{\mathrm{P}}(q,x) = \arg\max_x \sum_j s_j(x) q'_j.$$

5.2.1 不改变量化器的分解

高、低四位各自选择独立的坐标分块。写成 $z = 16h + l$，其中 $h, l \in \{0, \ldots, 15\}$，且 $\mathcal{C}_m[z] = (A_m[h], B_m[l])$。为两部分分别定义查找表：

$$T_m^{\mathrm{hi}}[h] = \|q'_{m,\mathrm{hi}} - A_m[h]\|_2^2, \qquad T_m^{\mathrm{lo}}[l] = \|q'_{m,\mathrm{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

平方距离可以沿这两个分块分解：

$$T_m[z] = T_m^{\mathrm{hi}}[z \gg 4] + T_m^{\mathrm{lo}}[z \,\&\, 15]. \tag{5.5}$$

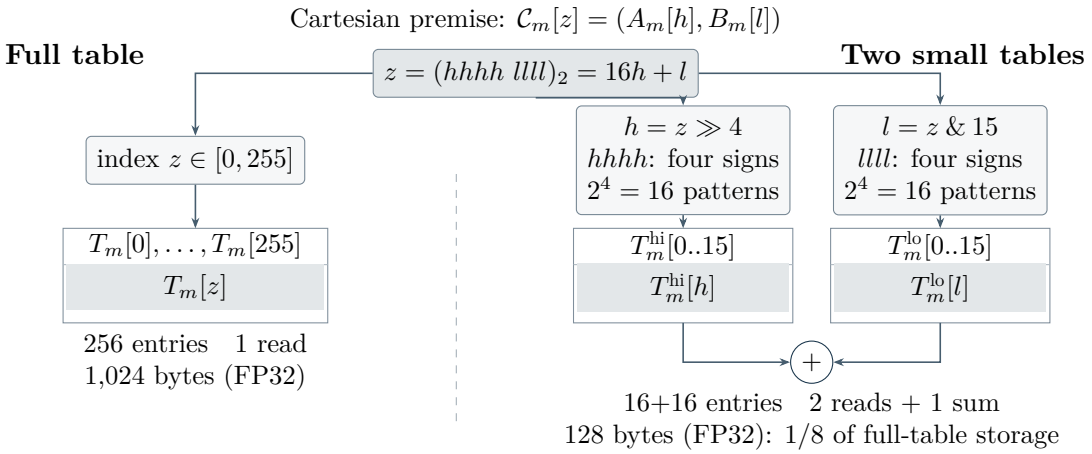


图 5.2. 主码 LUT 的 Cartesian 分解.

每个 byte 从索引一张 256 项表变为索引两张 16 项表，使表存储减为八分之一。该恒等式在实数算术下成立，FP32 求值则可能改变加法的结合顺序。构建时会检查码字 $16h, l$ 是否对应预期坐标分块；仅把学习得到的 PQ 码本标签切成两个 nibble，并不能保证这一结构。

5.2.2 分组、构造工作量与资源成本

更一般地，以 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ 将八个坐标分成连续组，使位字段对应 Cartesian 数字。当 $D_s = 8$ 时，每组有 $2^{8/g_{\mathrm{LUT}}}$ 项。每子空间的总项数及每查询的 FP32 字节数为

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

独立构建表项需要每子空间计算 $D_s 2^{8/g_{\mathrm{LUT}}}$ 个坐标贡献。两组只需整表八分之一的存储和十六分之一的坐标工作量。表 5.2 还列出合并查得值的成本；这些代数计数不预测整个查询的加速。

表 5.2. 八位、八坐标主码子空间的 Cartesian 表权衡。合并加法是连接查得值所需的代数最小值，不含跨子空间累加；不是实测指令计数。

g_{LUT}	项数	FP32 字节	查表数	合并加法	构造项数
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

表放在哪里取决于表与选择缓冲的合计占用。表先在全局工作区构建；完整分配不超过每 block 上限 $S_{\max}$ 时，再协作复制到共享内存。缓冲容量为 C_{buf} 时，普通及分组扫描分配为

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

对已评测路径，在没有可选融合精排或提前退出临时区时，启动规则为

$$\text{shared lookup} \iff S_{\mathrm{path}} \le S_{\max}.$$

扫描实验使用 $c = 1{,}000$、512 或 1,024 个线程的 block，以及 $C_{\mathrm{buf}} = 2{,}048$。整表占 96–384 KiB，两组表占 12–48 KiB。NVIDIA 对计算能力 12.0 给出的每 block 上限为 99 KiB [24]。最小整表加入选择缓冲后也超过上限：$M = 96$ 时，普通路径需要 114,696 bytes。最大的两组分配在分组路径中为 65,560 bytes。按启动规则，这些占用使整表留在全局显存，而两组表进入共享内存。[1]

四组与八组每子空间都只需 16 项，仍比两组少一半，但查表与合并次数更多。小表能为更多并发 block 留出资源；提取更多位字段、合并其值及为每个扫描组暂存副本又会增加工作。小表思想已有先例 [1, 2, 33]；本文将其结合到 JHQ 的两级 GPU 搜索中。第7.3节 比较各表形式的吞吐及逐候选 FP32 差异。

5.3 打包编码与候选并行内存访问

主码扫描将相邻候选分配给相邻线程。原有 byte 布局已将同一子空间的编码放在连续地址；打包进一步让每个线程通过一次 word 读取取得四个子空间编码，减少读取操作数。

5.3.1 地址映射与四字节打包

在列表 ℓ 中，行 r 的物理位置为

$$p = o_\ell + r, \qquad \mathrm{offset}(Z[m, p]) = mN + p.$$

[1]该放置由源码规则与公开上限推得；存档测量没有保留设备属性查询。

固定 m 时，$\Delta\,\mathrm{offset} = \Delta p$，所以相邻候选拥有相邻编码 byte；跨列表边界时可能失去这种相邻性。打包布局将同一候选的四个子空间编码组合起来：

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

移位和掩码可以恢复各个原始八位值：

$$(\mathsf{w}[j,p] \gg 8t)\ \&\ 255 = Z[4j+t,p].$$

编码的高、低 nibble 随后索引 式 (5.5) 中的两张表。

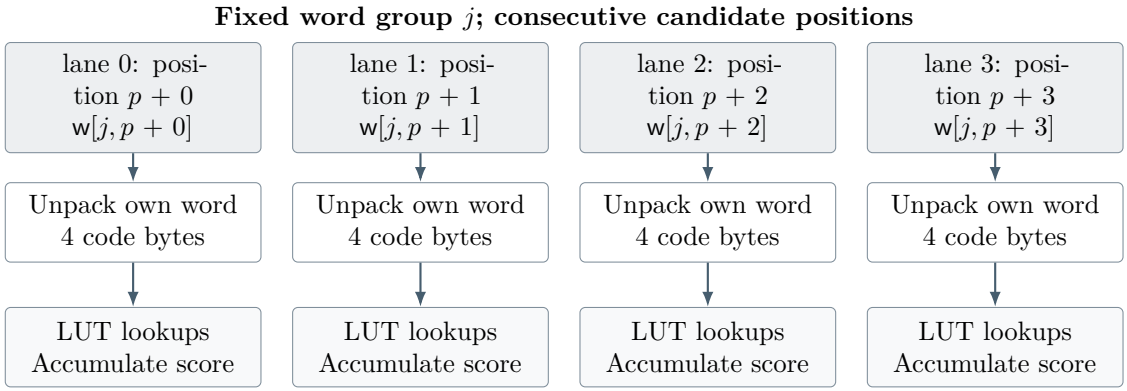


图 5.3. 候选并行的打包访问.

图 5.3 中每个 lane 读取并解包自己的 word，各 lane 的候选分数彼此独立。构建将 $[M,N]$ bytes 转为 $[M/4,N]$ words，要求 $4 \mid M$。直接把原数组转换成 word 类型会将不同候选的 bytes 混在一起，因此需要单独重打包。读写字节量及临时存储为

$$R = W = NM, \qquad S_{\mathrm{old+new}} = 2NM.$$

这次重打包不计入搜索时间。

5.3.2 访存开销与评分一致性

一次 word 读取替代每候选四次 byte 读取。对 32 个 lane，逻辑 word 窗口覆盖 $32\times4 = 128$ 个连续 bytes；实际事务数取决于对齐和活跃 lane。每候选载荷仍为 M bytes，解包保留每个编码。两种被测试布局都按子空间组织候选，因而该对比没有单独分离转置或缓存效应。第7.3节 测量其集成吞吐变化。

5.4 多 block 扫描中的精确全局主码 Top-c

小 batch 可能没有足够的查询来占满 GPU。把一个查询的候选分给多个 block 能提供更多并行工作，但这些 block 必须共同保留全局最佳候选。因此，各 block 需要完整短名单的容量，再归并各列表 [18]。这里的精确性使用共同的分数与位置顺序；只按距离截断时，并列候选可能不同。

5.4.1 候选分区与确定性排序

对一个查询，按 probe 顺序将选定 IVF 列表串接为 n_c 个候选出现实例。假设每个被访问行只出现一次，且索引保持固定。对 $g_{\text{scan}} \in \{1, 2, 4, 8\}$，block (q, j) 扫描

$$U_j = \left[\left\lfloor \frac{j n_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1) n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

区间表示串接序列中的候选，而非数据库 ID。各区间大小最多相差一项，长列表可能跨越边界。这比按相同列表个数分配更能平衡候选数量。probe 前缀 offsets 定位每线程的首个列表，随后以游标推进至后续列表；本阶段只读取主码。

为了使局部和归并选择一致，相同分数按物理位置排序：

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

位置 $p(x)$ 在固定索引中唯一，也不依赖原子追加顺序。对非 NaN 分数，这个完整键可以确定所有候选的先后。最终结果的并列则使用数据库 ID。只要各路径计算分数和排序规则相同，改变分区就不改变选中的键。

5.4.2 精确局部保留与全局覆盖

以 $\text{Top}_c(V)$ 表示前 $\min(c, |V|)$ 个键，其中 $c = \max\{k, \lceil \alpha k \rceil\}$。所有全局优胜者都可能落在同一组，因而每组只给 c/g_{scan} 配额并不安全；每组须有保留至多 c 项的容量。下面的缓冲保证说明：压缩保留仍符合共享界的局部优胜者。共享界 β_t 在 第5.5节 介绍；不共享时，它是排在一切有效键之后的占位键，称为哨兵。

引理 5.1 (局部缓冲保留)**.**
令 V_t 为一个 block 扫描到 chunk t 为止已评分的候选，B_t 为该时刻任意压缩或过滤后的有效缓冲。假设完整键固定、追加容量充足且 top-c 压缩精确。令 β_t 为迄今观察到的完整共享阈值的最小值，包括初始顶部哨兵。则

$$\text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

不共享时，阈值为哨兵，缓冲包含整个前缀 top-c。

证明.
空前缀显然成立。令

$$E_t = \text{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\}.$$

将 V_t 扩展为 V_{t+1} 时，$\beta_{t+1} \preceq \beta_t$，且

$$\text{Top}_c(V_{t+1}) \cap V_t \subseteq \text{Top}_c(V_t).$$

因此，

$$x \in E_{t+1} \cap V_t \Rightarrow x \in E_t \subseteq B_t.$$

局部拒绝的新键已有 c 个前驱，故排名 $> c$。容量可容纳其余符合准入的到达项。对任意符合条件的优胜者 x，

$$\#\{y \in B : K_q(y) \prec K_q(x)\} \leq \#\{y \in V_{t+1} : K_q(y) \prec K_q(x)\} < c,$$

所以精确压缩保留 x。过滤移除 $K > \beta_{\text{seen}}$；运行最小值覆盖陈旧读取。上述步骤通过归纳保持不变量。 □

一旦保留列表覆盖全部全局优胜者，就能从中恢复精确的 top-c。下一命题还允许每次归并只留下前 c 项。

命题 5.1 (覆盖与截断归并)**.**
在共同全序下固定键，令 U 为已访问空间，V 为任意保留子集。若 $\mathrm{Top}_c(U) \subseteq V \subseteq U$，则 $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$。成对有序归并并截断为 c 可保持该结果。特别地，对不相交分组，

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

该陈述也包含候选空间小于 c 的情形。

证明.
每个全局优胜者都在 V 内，而 V 的其他成员均排在全局优胜者之后。因此，从 V 选择可找回它们。对独立分组这一特例，设 x 属于全局 top-c，且位于组 U_j。全局上排在 x 前面的候选少于 c，所以在 U_j 内排在它前面的也少于 c。于是 $x \in \mathrm{Top}_c(U_j)$，局部结果的并集包含所有全局优胜者。取并集前 c 项，即按相同顺序精确找回全局优胜者。若整个空间少于 c 个成员，则全部保留。
归并时，两个有序列表的前 c 项就是并集的 top-c。被丢弃候选在该并集中已有至少 c 个前驱，无法进入任何更大并集的 top-c。对归并树归纳即可得到结论。 □

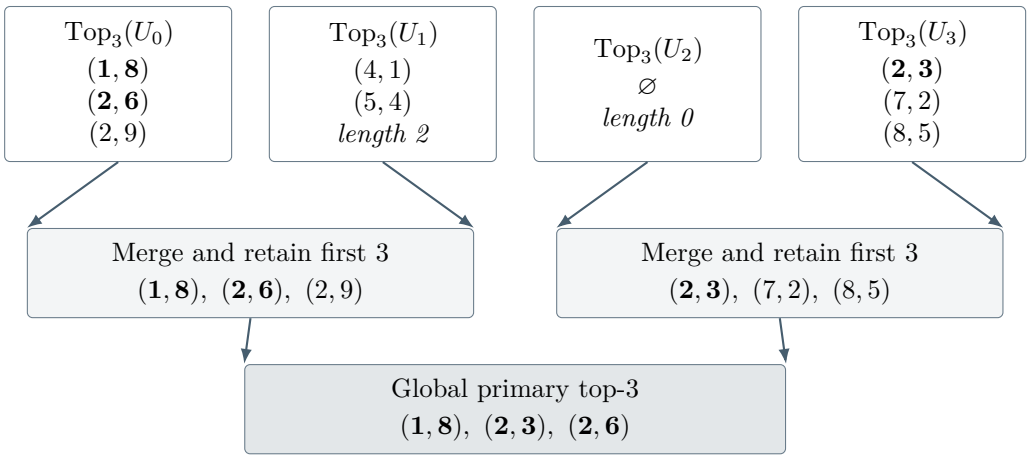


图 5.4. 局部覆盖与截断成对归并，示例取 $c = 3$、$g_{\mathrm{scan}} = 4$。条目是示意的 (D_{P}, p) 键，粗体为全局优胜者。短组 / 空组贡献有效前缀。归并按相同顺序最多保留 c 项。

每个 block 在共享内存中保存容量为 C_{buf} 的键对缓冲，该容量为二的幂。对 T_{blk} 个扫描线程，预留 $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ 项，容纳保留键及下一个 chunk。bitonic 压缩排序缓冲并最多保留 c 个有效键。有 c 个键后，最差保留键成为局部阈值 λ_j；此前阈值为哨兵。新候选只有在 $K \prec \lambda_j$ 时进入。这 c 个已保留键使候选继续到达时阈值仍然有效。

压缩之后，一个 chunk 每线程最多添加一个键，因此

$$|B| + |\Delta V| \leq c + T_{\mathrm{blk}} \leq C_{\mathrm{buf}}.$$

这给出 引理 5.1 的追加容量前提。结合 命题 5.1与5.2，便将缓冲实现连接到全局覆盖。写出全部主码分数需要 $O(n_c)$ 的存储和写入；扫描中直接保留候选则输出 $O(g_{\mathrm{scan}}c)$ 个键对。寄存器选择也是另一种方案 [18]。

5.4.3 归并实现、存储与边界情况

对 $g_{\text{scan}} > 1$，一个 256 线程 block 为每查询执行 $\log_2 g_{\text{scan}}$ 轮归并。长度为 a, b 的一对列表通过 Merge Path 切片输出 $\min(c, a+b)$ 个键 [14]。比较遵循 式 (5.10)，轮间使用 block 屏障。图 5.4 展示截断为何安全。除划分归并工作的搜索和同步开销外，内部节点最多输出 $c(g_{\text{scan}} - 1)$ 个键对。

两块缓冲在各轮间交替使用，形状为 $[b_{\text{cap}}, g_{\text{scan}}, c]$ 与 $[b_{\text{cap}}, g_{\text{scan}}/2, c]$；其中 b_{cap} 为预留查询容量：

$$S_{\text{pairs}} = 8 b_{\text{cap}} c \left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2} \right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

计数另占 $4 b_{\text{cap}} g_{\text{scan}}$ bytes，已预分配的共享界另占 $8 b_{\text{cap}}$ bytes，候选和其他工作区另计。这些分配在图捕获前扩容，之后持续保留以复用。归并 block 的共享内存保存列表长度，而非全部键。

短组或空组只贡献有效前缀。排序与输出使用不同补位键：

$$K_{\text{sort pad}} = (+\infty, \text{INT_MAX}), \qquad K_{\text{output pad}} = (+\infty, -1).$$

有效位置非负且小于 INT_MAX；负输出位置表示未使用条目。空组计数为零，$g_{\text{scan}} = 1$ 无需归并，直接写出。

单 block 路径 $g_{\text{scan}} = 0$ 最多保留 c 个最小计算分数，但截断处完全并列时不固定保留哪些候选。对非空 V，令 $h = \min(c, |V|)$，d_h 为其第 h 个分数：

$$\{x : d_x < d_h\} \subseteq B \subseteq \{x : d_x \le d_h\}, \qquad |B| = h.$$

因此，所有严格更小的分数都被保留，保留项也都不大于截断值；并列候选身份可以变化。评测构建关闭了可选的配额式近似选择器。第7.4.1节 通过 CPU 排序 GPU 已计算分数，检查这一距离顺序约束及分组完整键排序。

5.5 共享完整键阈值与执行调度

独立分组最多维护 $g_{\text{scan}} c$ 个条目，却只需找出 c 个优胜者。共享安全上界能减少这项维护。该界是包含并列位置的完整键：一组持有 c 个不同且已完整评分的候选后，其最差键便是全局截断键的上界；其他组可以拒绝排在该界之后的键。调度器另行选择组数。两个机制均使用固定精排预算，预算由 第6章 的方法选择。

5.5.1 具有显式见证的完整键上界

当 $|U| \ge c$ 时，令 $K_{(c)}$ 为全局第 c 个键。一组已完整评分的 c 个候选称为见证，因为它们的存在足以支持一个上界。若最大键为 τ_j，则 $K_{(c)} \preceq \tau_j$，因为已有至少 c 个候选排在它之前或与之相同。共享界初始化为顶部哨兵，并通过下式收紧：

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

少于 c 个有效候选的组不发布界。若整个候选空间少于 c，所有有效候选仍可进入。下一命题保证，即使各组读到较早的更新，按完整发布键过滤也不会丢失全局优胜者。

命题 5.2 (见证阈值下的安全拒绝)**.**
假设键固定，发布值由 c 个不同候选支持，读取返回初始哨兵或某个完整的已发布键。仅在 $K_q(x) \succ \tau$ 时拒绝候选，无论发布时序如何，都不会移除全局 top-c 候选。

证明.
一次发布具有含 c 个不同见证的集合 W_τ：

$$\max_{x \in W_\tau} K_q(x) \preceq \tau \quad \Longrightarrow \quad K_{(c)} \preceq \tau.$$

哨兵读取及已发布键的最小值满足同一不等式。因此

$$x \in \mathrm{Top}_c(U) \implies K_q(x) \preceq K_{(c)} \preceq \tau_{\mathrm{seen}} \implies x \text{ is not rejected.}$$

陈旧读取只会放宽阈值。先前安全拒绝后的发布仍然有效：保留的任意 c 个不同候选仍提供 c 个见证，无论它们是否是该组最佳的 c 个。 □

结合 命题 5.1 的覆盖条件，可以同时应用局部与共享过滤：

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\mathrm{seen}}, \tag{5.15}$$

其中 λ_j 是局部压缩阈值，τ_{seen} 是最后读取的共享键。局部比较严格小于，共享比较保留等号，因而也保留提供该界的候选。

5.5.2 发布、过滤与实现限制

GPU 将完整键编码为一个 64 位整数。对非负 FP32 分数、规范化正零及有效非负位置，

$$\kappa(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.16}$$

在此非 NaN 定义域中，整数比较与先比较分数、再比较位置的顺序相同：

$$(D_1, p_1) \prec (D_2, p_2) \iff \kappa(D_1, p_1) < \kappa(D_2, p_2).$$

有限键排在无穷和全一哨兵之前，无效位置不能发布界。

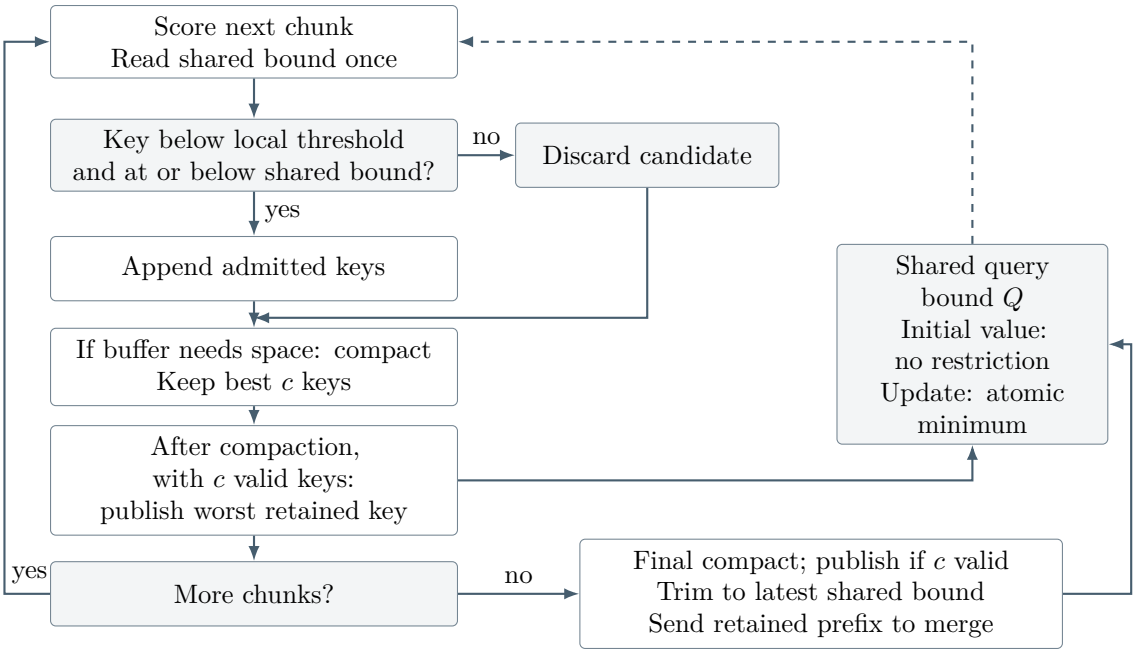


图 5.5. 共享键过滤与 chunk 循环.

每个扫描 chunk 由指定线程读取一次共享界。block 先计算完整主码分数，再准入符合条件的键，需要空间时压缩。压缩后持有 c 个有效键，便可用原子最小值发布最差保留键。最后一个 chunk 后，再次压缩并按最新共享界裁剪，得到送往归并的前缀。较早读取可能多保留一些候选，精确归并会移除它们。重置、扫描和归并在同一 GPU stream 中依次执行，各组不互相等待。

证明要求对齐的完整键读取。生成的 64 位读取及原子更新支持 PTX 内存模型下的这一前提 [26]，C++ 的 `volatile` 本身不提供该保证 [25]。[2]如 图 5.5 所示，过滤发生在完整主码评分之后。它节省缓冲和归并维护，却增加共享界读取与原子更新；第7.4节 测量这一权衡。

5.5.3 不改变预算地选择并行度

对 b 个查询使用 bg_{scan} 个 block 能提供更多并行工作，但每组也需要查表、缓冲和归并输入。调度器需要平衡并行度与重复工作。定义 block 大小、设备线程容量及候选数量的变化程度：

$$T_{\text{blk}} = \text{scan threads/block}, \quad H = n_{\text{SM}} T_{\text{resident/SM}}, \quad v = \frac{\text{sd}(n_c)}{\text{mean}(n_c)}.$$

填充规则在 batch 只使用部分设备容量时增加组数；另一规则在查询工作量不均时增加组数：

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

偏斜规则可覆盖填充上限。评测使用 512 个数据库向量作为伪查询，在固定路由下估计 v，无需 GT；这与第 6 章的输出一致性校准不同。后续搜索直接使用已存统计量与 batch 大小，不额外回读主机。常数由四个试验配置拟合，再用于更多配置的测试。一次性调度校准不计入稳态计时，记录的启动时间见 第3.5.2节。

选择一组时，在相同索引、probes 和预算下分派 $g_{\text{scan}} = 0$。其截断并列行为见 第5.4节；集成收益与调度损失在 第7.6节 评估。

5.5.4 观测到的执行路径

batch 大小解释了哪条路径支撑主要跨系统结果。对 1,000 个查询、512 个扫描线程，两组便已超过填充目标；RTX 5090 有 170 个 SM，每 SM 支持 48 个 warp [24]：

$$1000 \cdot 2 \cdot 512 = 1{,}024{,}000 > 0.6 \cdot 170 \cdot 48 \cdot 32 = 156{,}672.$$

因此填充规则选择一组，偏斜规则仍可选择四组。主比较中，38 条路由记录有 9 条超过 $v = 0.3$，全部是 Vogue（0.338–0.622）。Vogue 的 27 个配置使用四组，全部 105 个 arXiv / OpenAI 配置使用 $g_{\text{scan}} = 0$。后者获益于 LUT、打包、协作精排及 probe 选择，而非分组归并。Vogue 对比中，每 1,000 个查询有 3–15 个截断并列差异，没有非并列差异，最终输出相同；128 probes 下的并列范围为 7–11。

预算研究使用 128 条查询的校准样本与较大的留出 batch，路径计数为

	$G = 0$	$G = 2$	$G = 4$
held-out	10	0	2
$S = 128$	2	8	2

[2]检查使用 CUDA 13.0.88 的 `ptxas`、目标 `sm_120`，涉及 `ld.volatile.global.u64` 与 `atom.global.min.u64`；没有捕获驱动运行时 JIT 生成的原生指令。

十二个配置有八个换路径。收益同时取决于 batch 大小与工作量偏斜，并无固定 batch-256 边界。启发式未建模驻留、压缩和归并成本，伪查询统计也可能过时。其遗憾值是已测最佳固定组数的加速比除以调度器加速比，最差观测为 batch 256 时的 1.217。上述测试中精排预算固定，精排机制见 第5.6节。

5.6 Lane 协作残差评分与最终 Top-k

主码筛选将坐标工作量从 $O(n_c d)$ 降为 $O(cd)$，但每个保留候选仍需完整的残差距离计算。让多个 lane 共同处理一个候选可以缩短各 lane 的循环。本节固定预算，只改变这些工作的分配方式。

5.6.1 重建评分与仅依赖查询的常数

存储校正项 $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ 将主码与残差贡献组合为精排分数。在实数算术下展开，可看出该分数实际排序的对象：

$$\begin{aligned} D_{\mathrm{H}}(q,x) &= D_{\mathrm{P}}(q,x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) && (5.19)\\ &= 2\|q'\|_2^2 + \|\widehat{p} + \widehat{r}\|_2^2 - 2\langle q', \widehat{p} + \widehat{r} \rangle \\ &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. && (5.20) \end{aligned}$$

增加的查询范数对全部候选相同，因此排名等同于重建平方距离 [16]。标量执行先累加残差平方差，再加主码分数与校正项；查询及能够放入共享内存的码本在共享内存中暂存。

5.6.2 Lane 到 word 的映射与部分和归约

以 $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$ 表示分配给一个候选的 lane 数。各 lane 按步长访问残差编码 word：

$$d_{\mathrm{word}} = 32/B_R, \qquad \text{words of lane } a:\ a, a+L, a+2L, \ldots .$$

一个 word 包含四个八位码或八个四位码。各 lane 先累加自身解码贡献，再通过蝶形 shuffle，即一系列成对 lane 交换，合并部分和。子组 lane 零加上主码分数与校正项并写出结果。

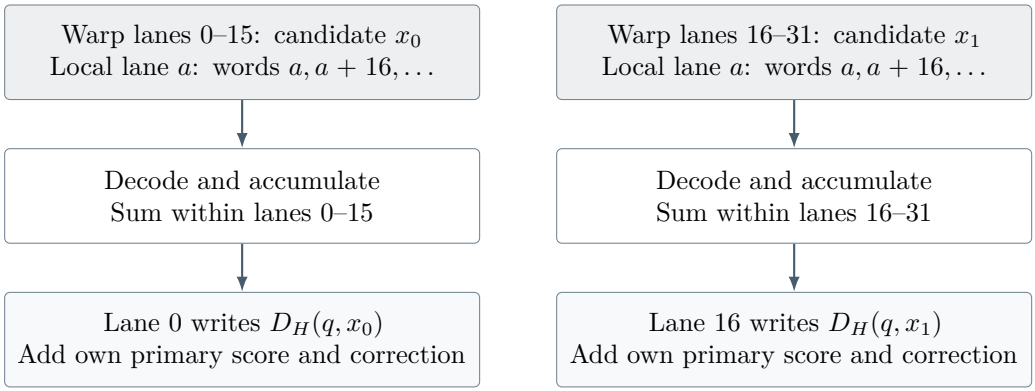


图 5.6. 十六 lane 残差精排.

十六 lane 时，一个 warp 独立处理两个候选，如 图 5.6 所示。局部 lane a 访问 words $a, a+16, \ldots$，warp 的 lane 0 和 lane 16 分别写出结果。行按 word 对齐且满足 $d_{\mathrm{word}} \mid D_s$，保证每个 word 不跨码本。所有 lane 都参加 shuffle，无效槽位贡献零，并在最终获得无穷输出分数。

实验名称对应这一映射：L0 用单 lane 读 bytes，L1 用单 lane 读 words，L8 / L16 / L32 则为每个候选使用对应数量的 lane。更多 lane 缩短局部循环，但增加 shuffle 工作，并减少同时处理的候选数。第7.3节 对比这些选择及其数值影响。

5.6.3 最终选择与数值边界

最终选择每一轮扫描全部保留槽位。假设至少有 k 个有限、有效且 ID 唯一的保留候选，每轮选择

$$i^* = \arg\min_{i<c}(D_{H,i}, \mathrm{ID}_i), \quad d^* = D_{H,i^*}, \quad D_{H,i^*} \leftarrow +\infty$$

并输出 (ID_{i^*}, d^*)。将该分数替换为无穷，使下一轮选中下一候选，从而得到保留池内精确的 top-k，无需每线程配额。有限项不足时，内核可能在无穷分数处重复最小有效 ID；全无效池返回无效 ID。评测配置满足有限项前提。

主码并列按物理位置，最终并列按数据库 ID。最终选择对输入分数精确，而路由和量化重建仍决定原向量召回率。改变求和顺序可改变 FP32 舍入，并重排近并列结果。

5.7 IVF 探测列表选择

路由须从数千个质心中选择少量 IVF 列表。对全部质心分数排序，比只寻找所需前缀做了更多工作。实现先用 radix 分趟确定截断值，再只排序被选中的列表。它在主码扫描前选择列表，与选择向量保留候选不同。

对质心 μ_i，计算路由分数及其并列顺序为

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

去掉共同查询范数不改变质心排名。不同于主码平方距离，这些分数可以为负。选择器返回

$$n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}}), \qquad I = \mathrm{Top}_{n_{\mathrm{sel}}}^{(s_i,i)}\{0, \ldots, n_{\mathrm{list}} - 1\},$$

以及选定列表长度的前缀 offsets。在 OpenAI 上，评测旋转与质心 GEMM 使用 TF32 乘法和 FP32 累加；这里的精确性针对这些计算分数，精度检查见 第7.2.3节。

5.7.1 Radix 选择与确定性排序

一个 block 由最高有效字节至最低有效字节执行四趟 radix。直方图统计具有已选前缀的键；下一个字节选择首个达到剩余目标排名的桶。对该直方图 H_t 及排名 r_t，其中 $r_0 = n_{\mathrm{sel}}$，

$$b_t = \min\Big\{b : \sum_{a\le b} H_t[a] \ge r_t\Big\}, \qquad r_{t+1} = r_t - \sum_{a<b_t} H_t[a].$$

重复此步骤，在四个 bytes 后确定截断键。为使整数的字节比较遵循带符号浮点分数顺序，先将带符号零规范化为正零，再映射分数位 u：

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

对非 NaN 分数，负数、非负数及异号三种情况给出

$$\begin{aligned} &D_1 < D_2 < 0 \Rightarrow u_1 > u_2 \Rightarrow \sim u_1 < \sim u_2, \\ &0 \le D_1 < D_2 \Rightarrow u_1 < u_2 \Rightarrow (u_1 \mid 2^{31}) < (u_2 \mid 2^{31}), \\ &D_1 < 0 \le D_2 \Rightarrow F(u_1) < 2^{31} \le F(u_2). \end{aligned}$$

该映射处理负值、无穷及零，而 式 (5.16) 只适用于非负分数。各趟按相同的融合乘加舍入方式重算分数，因此截断检测和收集阶段比较的是相同计算键。

各 warp lane 有自己的直方图副本以减少冲突，合计需要

$$256 \text{ bins} \times 32 \text{ lane replicas} \times 4 \text{ bytes} = 32\,\mathrm{KiB}.$$

这些副本仍需归约，并可能在 warp 间产生竞争。确定截断键 h 后，收集阶段区分小于它与等于它的键：

$$A_< = \{i : F(u_i) < h\}, \quad A_= = \{i : F(u_i) = h\}, \quad m = n_{\text{sel}} - |A_<|.$$

保留 $A_<$ 及 $A_=$ 中 ID 最小的 m 项。有序 warp ballot 用位掩码记录 lane 决策，结合前缀计数，使选择不依赖原子到达顺序。对选中集合 A，各成员的精确排名决定输出槽位：

$$\text{rank}(i) = |\{j \in A : (s_j, j) \prec (s_i, i)\}|, \qquad \text{out}[\text{rank}(i)] = i.$$

最后，列表长度形成 probe 前缀 offsets。若请求 probes 多于列表数，补位项长度为零：

$$i_j = i_{n_{\text{sel}}-1}, \quad L_j = 0 \quad (j \geq n_{\text{sel}}), \qquad \omega_{j+1} = \omega_j + L_j.$$

重复最后一个列表 ID 无害，因为补位长度为零，不会再次扫描；要求 $n_{\text{list}}, n_{\text{probe}} > 0$。

5.7.2 成本与独立正确性约束

Radix 各趟及收集阶段反复读取分数。选定列表的排序需要 $O(n_{\text{sel}}^2)$ 工作及随之增长的共享存储，所以选择规模很小时仍要支付固定直方图开销。第7.4节 比较吞吐并检查并列、带符号零、无穷、极值及超量 probes。FP16 近似与跨进程波动在那里另行检查；确定性选择器并不能消除其输入分数的差异。

第 6 章工作负载级预算校准

固定精排预算未必适合每种负载：全池校准在已测配置中选出的倍率从 4 到 200 不等（表 7.8）。新负载可能没有 GT 可供调参。本章回答 RQ3：在一组无标签查询上，检查较小预算是否改变较大参考预算返回的近邻，再将选定倍率复用于后续 batch。方法需要同时评估质量与校准成本。

6.1 精排预算与搜索成本

精排倍率控制路由后有多少候选继续接受精排。对访问空间 $U(q)$，JHQ 定义 [16]

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha),|U(q)|\}} U(q). \tag{6.1}$$

probe 选择决定 U，预算决定其中多少项通过主码筛选。分析假设 $|C_\alpha| \geq k$；候选不足时留有哨兵槽位。当 $k = 10$ 时，被测试网格覆盖 20–2,000 个槽位，无需重训索引。

图 5.1 高亮依赖预算的工作。路由候选和主码评分数量固定，但扫描中的准入与压缩工作可以改变。固定 batch 大小和路由时，逻辑成本模型为

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

其中 T_{base} 包括传输、旋转、路由和主码评分算术，其余项分别表示候选选择、残差评分与最终选择。资源变化使更小预算未必带来更短搜索，因此规则选择最小接受预算，而非实测最快预算（第7.5.5节）。每次试验复用训练索引，但仍重跑包括旋转、路由与主码评分在内的完整搜索。

在自适应 ANN 方法中的定位。 倍率 α 扩展保留候选集合，作用类似 Faiss 的 `k_factor` 及 cuVS 精排 [7, 10]。其他控制使用 GT 调参 [9]、学习式逐查询停止 [19] 或预测召回率 [6]。本文与参考输出比较一致性，为整个负载选择一个可复用预算。精排仍对重建向量评分，cuVS 示例则使用原始向量。

6.2 输出一致性与预算选择

校准需要一种不依赖近邻标签的比较对象，这里采用同一训练索引在较大预算下返回的 ID。令 $\mathcal{R}_\alpha(q)$ 包含 k 个有效、互异的返回 ID，并写成 $\mathcal{R}_{\mathrm{ref}}(q) = \mathcal{R}_{\alpha_{\mathrm{ref}}}(q)$。测试网格和参考预算为

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\mathrm{ref}} = A[0] = 200. \tag{6.3}$$

对同一组 S 个查询 $\mathcal{Q}_S$，忽略结果顺序和距离，计数小预算未能返回的参考 ID：

$$\begin{aligned} D_S(\alpha) &= \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\mathrm{ref}}(q)|\right) \\ &= \frac{1}{2} \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_\alpha(q) \triangle \mathcal{R}_{\mathrm{ref}}(q)|. \end{aligned} \tag{6.4}$$

接受规则 $D_S(\alpha) \leq \epsilon = 1$ 表示整个样本最多缺失一个参考 ID。目标是找到通过该规则的最小网格预算：

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \leq \epsilon\}. \tag{6.5}$$

没有较小预算被接受时，使用参考预算 200。第6.3节 说明二分何时能精确找到这一最小值。

由于容许量始终只有一个 ID，增大样本会降低允许缺失的参考答案比例。当 $k = 10$ 时，

S	32	64	128	500
$\epsilon/(Sk)$	0.003125	0.0015625	0.00078125	0.0002

大样本既覆盖更多查询，也施加更严格的相对阈值。第7.5节 中独立的测试查询评估从校准池选出的预算能否保持质量。

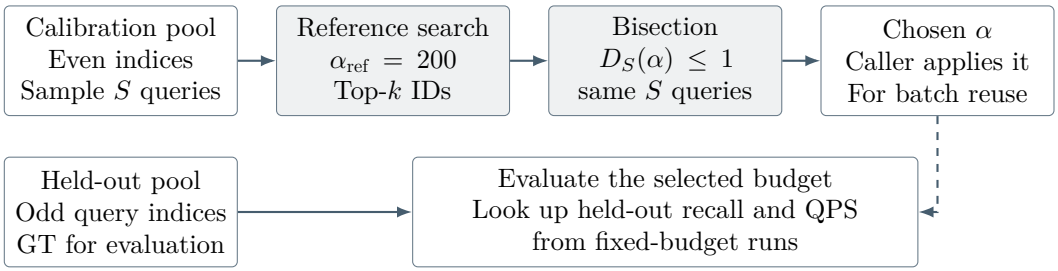


图 6.1. 工作负载级校准与评估.

图 6.1 中参考搜索和试验搜索使用同一组 S 个查询。GT 在预算选择后、于留出查询上评估质量，所选预算的吞吐来自固定预算测量。第3.5.2节 定义这些计时包含的调用。

6.2.1 一致性控制什么

当保留候选嵌套且最终排序固定时，不一致有直接解释：它计数被主码筛选排除的参考答案。在 命题 6.1 下，

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

对任意含 k 个元素的输出和 GT 集合 $\mathcal{T}_k$，GT 命中损失不会超过缺失的参考 ID 数：

$$|\mathcal{R}_{\text{ref}} \cap \mathcal{T}_k| - |\mathcal{R}_\alpha \cap \mathcal{T}_k| \le |\mathcal{R}_{\text{ref}} \setminus \mathcal{R}_\alpha|.$$

在样本上求和再除以 Sk，得到

$$\begin{aligned} r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}, \\ D_S(\alpha) \le \epsilon \implies r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{\epsilon}{Sk}. \end{aligned} \tag{6.7}$$

这是校准样本上相对参考的召回率界。参考本身的误差仍由 GT 评估（第7.8.2节），该界不会自动迁移至未见查询。

6.2.2 抽样分辨率与有序检验

小样本可能漏掉不常发生的返回 ID 变化。下面的模型说明，这些变化能以多高概率仍未被观察到。首先，令 H_S 计数缺失任意参考 ID 的查询数。由于 $H_S \le D_S$，被接受预算满足 $H_S \le 1$。对预先指定的预算及独立同分布查询，以 u 表示该查询事件概率，容许一次事件的单侧 95% 二项上限满足 [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

更强的模型将每个返回近邻位置视为概率相同的独立事件。把上限应用到全部结果槽位，得到下表第二行：

	$S = 128$	$S = 500$
i.i.d. query events	0.03652	0.009452
i.i.d. homogeneous result slots	0.003701	0.0009484

槽位模型给出约 $4.74/(Sk)$：$S = 500$ 时相当于千分之一左右的参考结果缺失率，$S = 128$ 时则约为千分之四。查询独立并不使各结果槽位独立。这些是模型下的参考不一致分辨率，而非重复使用固定池样本的保证。

只有每个预算检验具有有效的统计误差界时，单调性才允许按固定序列检验预算 [3]。实现中的一个 ID 接受阈值检查的是一致性，本身不认证未来查询的 GT 召回率。

有限池扩展。 对实际抽样方式，可以改为从校准池的构成计算接受概率。令 $X_q(\alpha)$ 计数查询 q 缺失的参考 ID，假设它在该执行区间内不随查询进入的子集或位置改变。池大小为 N_{cal}，令 $n_j = |\{q : X_q = j\}|$。均匀无放回样本仅在全部查询无缺失，或恰有一个查询缺失一个 ID 时通过 [23]：

$$\Pr\{D_S(\alpha) \le 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\mathrm{cal}}}{S}}. \tag{6.9}$$

越界二项系数取零。将接受概率转为 GT 损失预测，还需知道哪些预算通过及各预算的留出质量。第7.5.5节 将记录的子集和与这一计算对照。

6.3 预算搜索与单调性条件

6.3.1 二分与完整枚举

预算按由大到小排列，因而要找的是最后一个被接受的网格下标。记 $J = |A|$，先计算一次样本参考结果，再按以下步骤搜索：

1. 设置 $l = 0$、$h = J - 1$。零是回退项。
2. 当 $l < h$ 时，运行 $A[j]$ 并计算 $D_S(A[j])$，以测试 $j = \lfloor (l + h + 1)/2 \rfloor$。
3. 若 $D_S(A[j]) \le \epsilon$，设置 $l = j$；否则设置 $h = j - 1$。
4. 返回 $A[l]$，复用前应用它。

八点网格最多需要三次试验搜索及一次参考搜索。枚举则在同一样本上测试参考及另外七个预算。即使被接受预算之间存在被拒预算，枚举也能找到最小值；所有接受下标构成前缀时，二分保证找到该最小值。两种计时均包含参考搜索（第3.5.2节）。

6.3.2 单调性的一个充分条件

一个充分结构是短名单嵌套：较小预算保留的每个候选也出现在较大预算中。精排分数及最终并列顺序固定时，参考答案一旦通过短名单，就必然通过最终选择。下面命题精确表述这一点，并说明为何不一致随预算增大而下降。

命题 6.1 (嵌套保留集合保持参考成员资格)**.**
对一个查询，假设路由固定，且每当 $\alpha \le \beta \le \alpha_{\mathrm{ref}}$ 时保留集合满足 $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\mathrm{ref}}}$。假设至少有 k 个有效保留候选、精排分数固定、最终并列使用同一确定性全序。各结果均是该顺序下精确 top-k。则

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\mathrm{ref}}| = |C_\alpha \cap \mathcal{R}_{\mathrm{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

当上述条件对每个抽样查询都成立时，后一陈述成立。

证明.
对 $x \in \mathcal{R}_{\mathrm{ref}} \cap C_\alpha$，共同全序给出

$$|\{y \in C_\alpha : y \prec x\}| \le |\{y \in C_{\alpha_{\mathrm{ref}}} : y \prec x\}| < k.$$

故 $x \in \mathcal{R}_\alpha$。结合 $\mathcal{R}_\alpha \subseteq C_\alpha$，得到 式 (6.10) 中的交集恒等式。由嵌套性可得 $|C_\alpha \cap \mathcal{R}_{\mathrm{ref}}| \le |C_\beta \cap \mathcal{R}_{\mathrm{ref}}|$。对查询的缺失参考计数求和，得到 $D_S(\alpha) \ge D_S(\beta)$。 □

固定键下的精确主码 top-c 产生嵌套前缀。在命题的其他前提下，这使参考一致性检验单调；GT 召回率和运行时间未必单调。

表 6.1. 嵌套与非嵌套候选池.

Case	C_1	C_2	C_3	$\lvert C_i \cap \mathcal{R}_{\text{ref}}\rvert$, $i = 1, 2, 3$
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

表 6.1 给出 $k = 1$、精排顺序为 $a \prec b \prec c$、参考为 $\{a\}$ 的示例。嵌套池保持参考成员资格。当主码分数全部并列时，改变截断选择可使中间池排除较小池曾保留的答案，使重叠从 1 降至 0，再回到 1。

6.3.3 已评测分组路径的推论

同一校准样本内，S、路由及扫描线程数固定。调度器选择组数不使用 c，所以参考与试验预算使用相同扫描路径和 LUT 求和。固定 B_r、十六 lane 精排及最终 $(D_{\text{H}}, \text{ID})$ 排序，使每候选的精排分数与并列顺序独立于预算；分组选择进一步提供所需的嵌套主码短名单。

推论 6.2 (分组校准)**.**
对已评测分组配置，在第 4、5 章的表示与共享阈值约束下，各预算的主码保留集合嵌套。因此 命题 6.1 适用，二分与枚举返回相同的网格最小接受预算。

证明.
引理 5.1与命题 5.1与5.2 在共同键 $(D_{\text{P}}, \text{position})$ 下给出精确主码 top-c。其前缀随 c 增大而嵌套。固定精排 / 最终顺序满足 命题 6.1 的其余前提。 □

推论覆盖 2,176 个校准样本。另有 128 个样本，即 OpenAI-3072 的 $S = 128$ 情形，使用仅按距离的选择，截断并列可能破坏嵌套。记录实验的全部 2,304 条曲线单调，二分与枚举相同（第7.5.5节）。这验证了存档中的选择；分组路径的保证则来自推论。

全池覆盖。 两个全池配置对 Vogue 使用分组扫描，其余十个使用 $g_{\text{scan}} = 0$。全部十二个与枚举相同，但十个仅按距离的配置仍不在 推论 6.2 内：它们需要共同截断顺序，或没有跨越截断处的并列参考答案。校准和测试使用相同路径并不能建立嵌套。

基于排名的扩展。 在相同嵌套与固定分数前提下，存储每个参考答案的主码排名，就能知道各预算何时将其丢失：

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\text{ref}}(q)} \mathbf{1}\{\text{rank}_{\text{P}}(x) > c(\alpha)\}. \tag{6.11}$$

这样可以从导出的参考排名产生整条一致性曲线，代替重复搜索；目前尚未实现。

6.4 预算选择基线

对照区分三个问题：校准能否优于固定预算，二分能否与完整枚举选出同一预算，以及在可接受留出质量下还剩多少速度空间。表 6.2 列出各自使用的选择信息。固定预算比较共享索引、路由和留出查询，计时边界见 第3.5.2节。

表 6.2. 预算对照及回答的问题。留出 GT 用于评估每个结果，但只有诊断 oracle 使用它选择预算。一致性搜索的参考固定为 200。

对照	选择信息	问题
固定 100	不校准	相对继承默认值的收益?
固定 64 及更小网格预算	不校准	简单固定选择能否消除校准需求?
抽样二分	S 个无标签查询；返回 ID 一致性	低成本样本能否选出可复用预算?
同样本枚举	相同 S、参考及阈值；全部网格预算	二分是否漏掉网格最小接受预算?
全池校准	全部 500 个校准查询；相同二分规则	更广覆盖及更严格归一化容许量带来什么变化?
留出 Oracle	全部预算的 GT 召回及计时 QPS	哪个已测预算在诊断质量约束下最快?

对有标签诊断，以 $\widehat{r}, \widehat{Q}$ 表示留出召回率和实测 QPS，并令 $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$ 为测试网格上的最佳召回率。Oracle 选择损失不超过容许量的最快预算：

$$\alpha_{\text{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

这个 oracle 使用 GT 和计时，是评估基准；校准本身不使用这些标签。它的召回容差 δ 不同于一致性容许量 ϵ。一致性规则最小化被接受预算，oracle 则在质量约束下最大化速度。枚举检查预算搜索，全池校准在更严格比例 $\epsilon/(Sk)$ 下检验更广样本覆盖；固定 64/200 提供更小和更高预算对比（第7.5节与表 7.9）。

6.5 校准质量与复用成本

6.5.1 选择校准规模

为了分辨千分之一量级的参考不一致，第6.2节 的独立槽位模型提示

$$\frac{4.74}{Sk} \simeq 10^{-3}, \quad k = 10 \quad \Longrightarrow \quad S \simeq 474 \approx 500.$$

这为除小样本外测试整个 500 条查询的校准池提供了理由。该模型针对参考一致性；实测留出质量及抽样风险见 表 7.8与图 7.6。全池二分的搜索顺序条件仍为 第6.3节 所述条件。

6.5.2 风险与所选预算性能

令 m 个随机校准样本分别选择预算 $\widehat{\alpha}_r$。以最佳已测留出召回率为参考，定义第 r 个样本的损失及损失超过目标的比例：

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m} \sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

平均损失说明平均质量，越阈频率说明多常未达标，尾部损失则说明未达标时的幅度。表 7.9与图 7.6 报告这些汇总，逐样本最大损失见 第7.5.5节。

报告速率 $\overline{Q} = m^{-1} \sum_r \widehat{Q}(\widehat{\alpha}_r)$ 将各样本选中预算的既有实测吞吐取平均。这是基于固定预算运行的估计，不同于中位预算的速率，也不同于直接计时校准后的查询流。表 7.10 将其与质量一起报告。

6.5.3 复用成本

只有后续搜索节省足够时间，校准才能回本。令 T_{cal} 为一次性校准成本，T_{chosen} 为所选预算下每 batch 耗时，T_f 为固定预算下同样 batch 的耗时。对后续 B_{reuse} 个等大小 batch，

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}} T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}} T_f. \tag{6.14}$$

当 $T_{\text{chosen}} < T_f$ 时，用启动成本除以每 batch 节省时间，得到回本次数：

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

严格节省时间的条件直接由下式得到：

$$\begin{aligned} T_{\text{policy}} < T_{\text{fixed}} &\iff T_{\text{cal}} < B_{\text{reuse}}(T_f - T_{\text{chosen}}) \\ &\iff B_{\text{reuse}} > B^*_{\text{reuse}} \qquad (T_{\text{chosen}} < T_f). \end{aligned}$$

若 $T_{\text{chosen}} \geq T_f$，则无论多少 batch 都不能靠吞吐收回校准成本。每 batch 有 n 条查询时，$T = 1000n/Q$ 将 QPS 转为毫秒。当所选预算变化时，$1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$，取平均 QPS 的倒数会低估平均 batch 时间。计时边界和直接查询流检查见 第3.5.2节与第7.5.6节，模型估计见表 7.10。

每隔 H 个 batch 重新校准，会增加平均每 batch 的 T_{cal}/H 成本。自动检测工作负载变化及更新校准仍属后续工作。

6.6 校准复用与适用范围

复用假设索引、k、路由配置及查询工作负载稳定（第6.3节）。调用方须在后续搜索前应用选定的 α。第6.5节 定义质量和复用成本的评估方式。存档预算实验使用主评测版本，包括 OpenAI 上的 TF32 算术；FP32 敏感性检查未重新执行预算校准。

第 7 章实验评估

本章报告四个研究问题的证据。第7.2节 比较 JHQ-GPU 与其他 GPU 索引（RQ4）；第7.3节、第7.4节与第7.6节 评估面向表示结构的执行及其选择正确性（RQ2）；第7.5节 检验工作负载级精排校准（RQ3）；第7.7节 测量 GPU 构建与容量（RQ1）。第7.8节 考察数值敏感性，以及召回损失能够归因到哪些来源。

7.1 实验协议

7.1.1 比较协议与可复现性

比较采用 第3.2至3.4节 定义的数据、主机到主机计时和前沿规则。搜索吞吐不计构建与校准；每项执行消融保留自己的基线。表 3.4与3.5 对应实验家族、实现和结果位置，第3.5.2节记录计时约定。下文区分补充测试的基线是与 JHQ 一同重测，还是取自主比较存档。历史 CPU 结果独立报告于 第7.2.7节。

7.2 跨系统召回率与吞吐比较

7.2.1 比较范围与前沿构造

在召回率 0.97、batch 为 1,000 时，JHQ-GPU 在 Vogue 和两个 OpenAI 数据集上领先已测方法，在 arXiv 上与 IVF-RaBitQ 接近持平；四个数据集的吞吐均达到 CAGRA fp32 的 1.32–1.75×（图 7.1与表 7.1）。主比较使用 表 3.2 中的前四个数据集、共同查询与真实近邻，以及 第3.4节 的图、压缩 IVF 和暴力搜索基线 [17, 28, 33]。1,590 个有效测量覆盖每数据集 27–39 个 JHQ 配置，以及每基线家族 54–264 个配置。有限参数覆盖见 表 3.3与第3.4节。

执行路径随工作负载而变。全部 105 个 arXiv 和 OpenAI 配置使用每查询一个 block（$g_{\text{scan}} = 0$）；27 个 Vogue 配置因候选数变异系数为 0.338–0.622，采用四组（$g_{\text{scan}} = 4$）。因此只有 Vogue 的前沿包含分组保留、共享阈值和归并（第5.5节）。

前沿也比较了不同存储预算。九位 RaBitQ 与 JHQ 的名义编码大小均为每向量 $9d/8$ 字节，四位和八位变体则更少。表 4.1与第7.7节 在速度之外报告表示成本及实测分配。

7.2.2 相同召回率下的吞吐

对 CAGRA fp32 的领先在两种估计器下都成立，而 arXiv 与 RaBitQ 的微小差距会改变方向（表 7.1）。插值与满足阈值的实测估计器遵循 第3.4节。

表 7.1. 召回率 0.97 的比较。每个比值为“插值 / 实测”；大于一有利于 JHQ。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的小幅差异随估计器改变方向。

数据集	JHQ kQPS	相对 CAGRA fp32	相对 IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

Vogue 对 RaBitQ 的小幅领先依赖分组执行。在索引、probes、预算及最终输出固定时，batch 为 1,000，改用单 block 会使召回率 0.97 的 JHQ / RaBitQ 比从 1.059 降至 0.933。CAGRA int8 在召回率 0.90–0.92 快 1.20–1.98×，但其已测网格无法达到 0.97。RaBitQ 在部分高于 0.98 的目标领先；CAGRA fp32 在 arXiv 接近 0.99 时领先（图 7.1）。

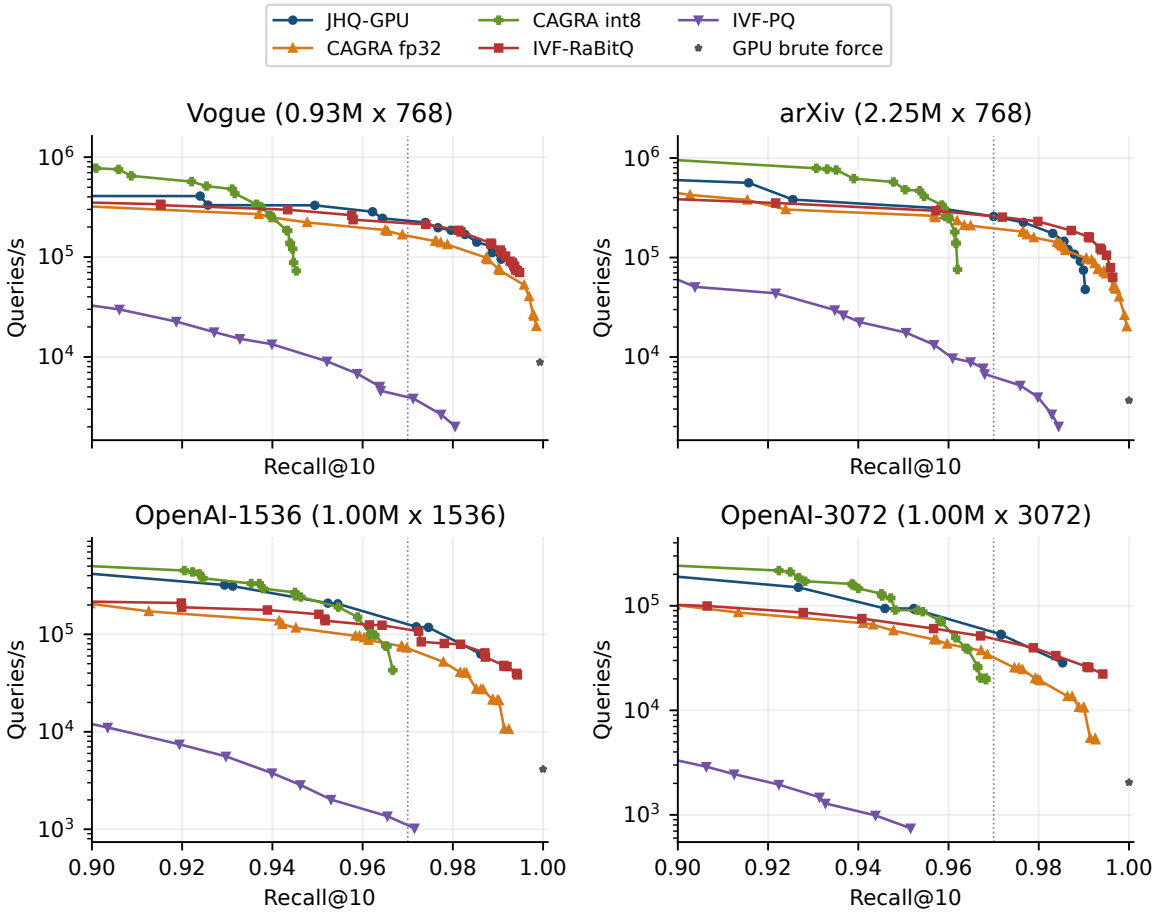


图 7.1. 四个主比较数据集上的实测主机到主机召回率—QPS 前沿。显示范围从 0.90 开始，0.97 处画垂线。包括 CAGRA int8 在内的所有基线家族均参与前沿构造。线连接实测点；吞吐轴为对数。

7.2.3 OpenAI 数据集上的精度敏感性

相同配置下，用 FP32 代替 TF32 使吞吐降低约 5%，已查 FP32 前沿仍领先 CAGRA fp32。源码检查发现，主实现对 OpenAI 旋转和质心矩阵乘使用 TF32 乘法、FP32 累加（第4.2节）。精度检查改变这一算术开关，在不同 probe 配置间交替执行两个变体：OpenAI-1536 为 37 个配置，OpenAI-3072 为 39 个，每个在 batch 为 1,000 下测五轮。两者复用同一 TF32 训练模型，改变 add 和查询算术，包括编码与路由；这不是独立的 FP32 重训。

表 7.2. FP32 敏感性。QPS 比为相同配置下 FP32 / TF32 的几何平均；召回差采用同一方向。末两列比较召回率 0.97 下的 FP32 前沿与主比较存档的 IVF-RaBitQ、CAGRA fp32 前沿。精度检查沿用主比较的基线测量值。

数据集	配置对	QPS 比	Δ 召回	相对 RaBitQ	相对 CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

相同配置下，召回提高 0–0.0005。召回率 0.97 时，FP32 比同轮测量的 TF32 慢约 3%，但对存档 RaBitQ 的比值在 OpenAI-1536 为 1.086，OpenAI-3072 为 1.079；对存档 CAGRA fp32 为 1.679 和 1.629。重测原始变体得到对 RaBitQ 的 1.118 和 1.114，历史值则为 1.130 和 1.157。图 7.2 展示成对精度比较；下一项检查重新测量基线。

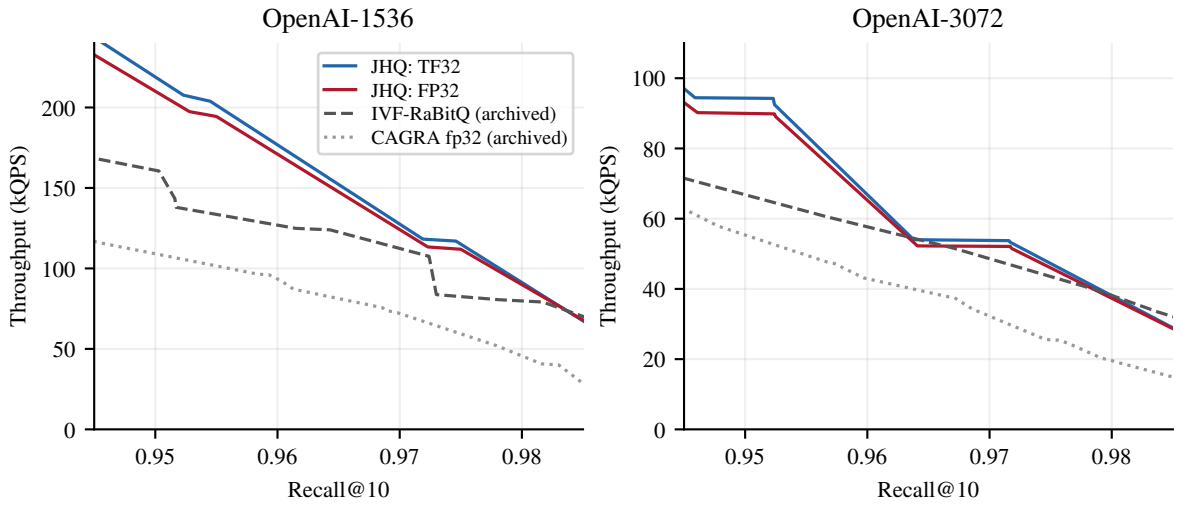


图 7.2. 精度检查。实线为一同测量的 JHQ 变体，虚线 / 点线基线来自主比较。均使用本章前沿与插值规则。

7.2.4 与 JHQ 一同重测的基线

同场次检查在其已测网格中，确认 FP32 在召回率 0.97 下按两种估计器均领先。表 7.3 报告对 RaBitQ 和 CAGRA fp32 的比值。

每个 OpenAI 数据集的十二个成对配置采用主比较的内存池设置。基线运行前后各有一次五轮 JHQ 重复，其中位速率按几何方式组合。相同配置 FP32 / TF32 的均值，OpenAI-1536 为 0.956，OpenAI-3072 为 0.964。两变体均复用 TF32 训练缓存，只改变 add / 查询算术。

表 7.3. 同场次检查，召回率 0.97、batch 1,000。基线比值为插值 / 严格实测阈值；FP32 / TF32 为插值。所有方法一同重测；精度检查则沿用主比较的基线测量值。

数据集	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

检查只覆盖召回率 0.97 附近的有限网格。OpenAI-3072 的 RaBitQ 括区间跨越召回率 0.9669–0.9796，因此优势幅度依赖网格覆盖和估计器。这是局部比较的证据，并非独立 FP32 重训前沿或全部召回目标的证据。

7.2.5 解释 OpenAI 比较

两个 OpenAI 导出均来自 Qdrant 的 DBpedia 嵌入，因此可以用于比较维度差异 [31, 32]。解释维度变化前先检查了行对齐：截断并重新归一化后，全部 1,000 个查询行和 10,299 个抽样数据库行的余弦均为 ≥ 0.9980，中位数为 1.000000；真实近邻集合平均共享十个邻居中的 9.046 个。这些检查支持行对齐，但不构成原始实体 ID 的完整连接。维度翻倍后，对 CAGRA fp32 的比值仍为 1.747，对 RaBitQ 的比值从 1.130 变为 1.157，因此维度本身不能解释观察到的优势。

128 probes 下，较小的精排倍率伴随较大收益：OpenAI 为 4–8，Vogue 为 64，arXiv 为 100，而后续全池校准对 arXiv 选出 200。这是跨工作负载的相关关系，并非受控因果比较；第8.2.3节 检验一种表示层面的解释。小幅差距和测量敏感性见 第7.8节。

7.2.6 小 batch 改变竞争区间

batch 为 8、召回率为 0.97 时，CAGRA fp32 在两种估计器下都快于全部四个数据集的 JHQ。较低召回率时 JHQ 仍有竞争力，但接近持平的比较依赖如何使用实测端点。小 batch 研究在四个主比较数据集上，以 batch 为 8、32 比较调度 JHQ、CAGRA fp32 / int8 和 IVF-RaBitQ。五轮同步主机到主机测试分别使用 1,000、992 条查询。1,808 个测量包含单 block JHQ 对照；IVF-PQ 和暴力搜索不在这项研究中。

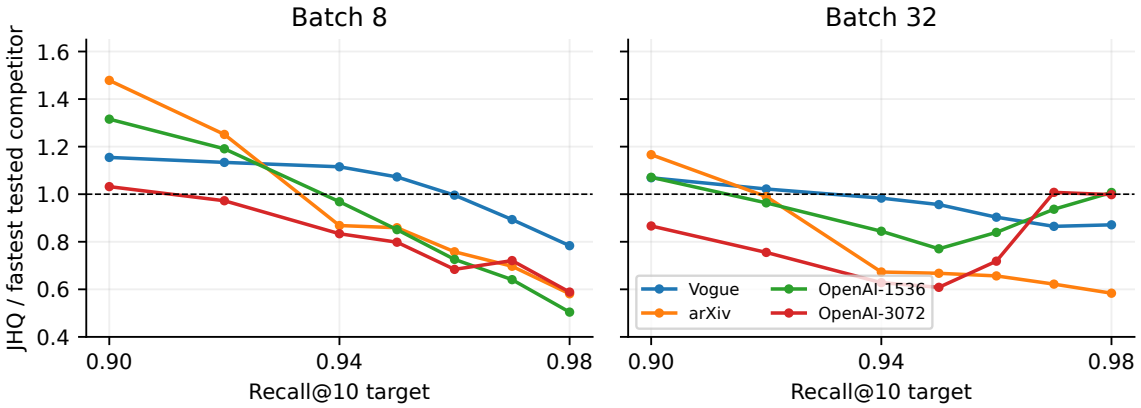


图 7.3. 小批量研究的 JHQ / 最快对手 QPS 比，采用保留合格实测端点的插值。大于一有利于 JHQ。严格阈值见 表 7.4。

表 7.4. 小批量研究 QPS 比：保留合格端点的插值 / 严格实测阈值。各估计器独立选择最快对手；满足阈值的更高召回端点可入选，无需外推。

数据集	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

召回率 0.97 时，插值得到的 JHQ / 最快对手比值，batch 为 8 时为 0.641–0.893，batch 为 32 时为 0.622–1.008。OpenAI-3072 在 batch 32 的实测阈值下有利于 JHQ，而 Vogue 在 batch 为 8、召回率为 0.95 时的领先依赖插值（表 7.4）。

召回率 0.90 时，包含达标端点的插值在 batch 为 8 时有利于 JHQ，比值为 1.032–1.479；严格阈值却只有三个胜出。batch 为 32 时比值为 0.866–1.166，严格阈值只有一个胜出。调度在召回率 0.95 时相对单 block 提升 1.7–3.3×，改善 JHQ 执行，但不能保证跨系统领先。

边界包含接口成本。仅含传输和同步的空调用中位数为 0.012–0.072 ms，搜索计时保留了这一成本。CAGRA 使用 Python，JHQ / RaBitQ 使用 C++；预热约定见 第3.5.2节。

7.2.7 与 CPU 实现的历史比较

较早的一次比较使用 JHQ 作者的 CPU 实现，在相同召回率下得到 GPU / CPU 吞吐比：Vogue 为 52–85，arXiv 为 56–98，OpenAI-1536 为 85–108，OpenAI-3072 为 75–77。测试在同一服务器的 Xeon Platinum 8470Q 上进行，并对 CPU 工件做了本地兼容性和 Recall@10 修复。图中的 CPU 前沿对 Vogue 和 OpenAI-3072 取 16、32 线程较快值，另外两个数据集

采用 32 线程。扫描范围为 $\alpha \in \{2, 4, 8, 16, 32, 100\}$ 及 128–1,024 probes；更小 probe 数受线程启动开销影响而排除。比值在相同召回率处按吞吐对数插值得到。

这些是历史实现之间的比较。GPU 使用较早的实现，CPU 码本单独训练，两个 OpenAI 的 CPU 索引有 4,096 个列表，而 GPU 为 8,192 个。多数 CPU 配置只计时一次。较晚的十月 CPU 精排研究也未匹配设置：Vogue 使用 1,024 个列表和 $[8, 4]$ 位，而主比较为 4,096 个列表。若要给本文评测 GPU 版本的硬件加速比，仍需匹配索引与计时条件。

7.3 Cartesian LUT、打包载入与 Lane 精排

7.3.1 LUT 分解与吞吐

两张 16 项表在 24 个配置中的 22 个是最快的已测形式，但实数算术中的分解不意味着 FP32 分数或最终 ID 完全相同。图 7.4 测试六个数据集、8、32、128、512 四种 probe 数，固定 $\alpha = 100$，并令 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$。五轮采用 batch 1,000，Stella 为 999。两个例外接近持平：arXiv 在 128 probes 下，一组快 0.23%；OpenAI-3072 在八个 probes 下，四组快 0.84%。两组对一组的配对比较赢得 24 个配置中的 23 个，各数据集几何平均比为 1.021–1.318。

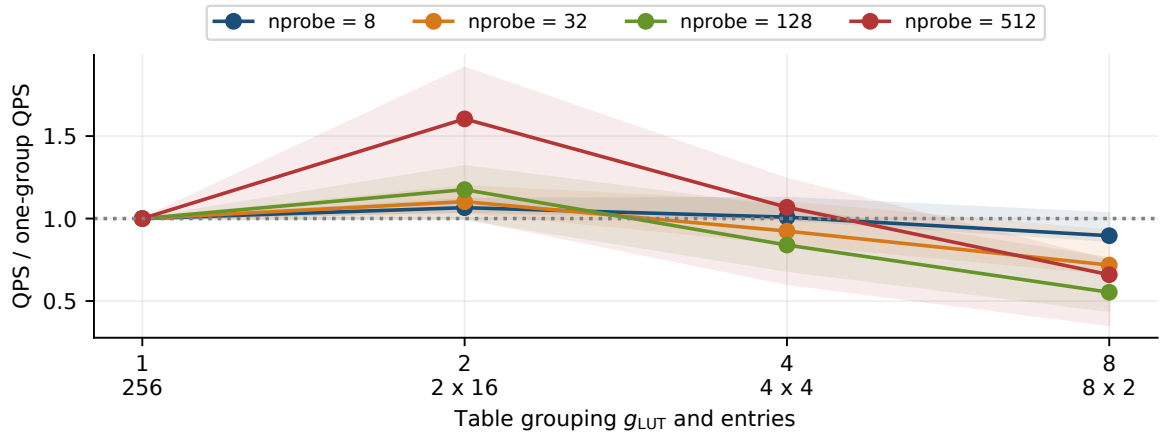


图 7.4. 固定 $\alpha = 100$ 下的 Cartesian 表分组。组数表示 g_{LUT}，不是并行扫描组。曲线为各数据集相对一组实验臂 QPS 比的中位数；阴影表示范围。

各配置的两组 / 一组比值为 0.998 到 1.914。四组和八组每个子空间均存 16 项，却需要更多查表和加法，较小的表并非总能补偿这些成本。占用率和指令成本未被单独分离。每个四变体配置内，最大报告召回差为 10^{-4}。

严格分数检查在 Vogue、arXiv 和 OpenAI-3072 上，固定 128 probes、$\alpha = 100$ 比较相同候选。整表与 $16 + 16$ 表有 22.6–26.1% 的分数 bit 模式不同，最大相对误差为 4.53×10^{-7}。全部表形式之间，20.5–29.4% 不同，最大误差为 4.63×10^{-7}。Vogue 的 256 条查询中有二至四条最终 ID 不同，arXiv 和 OpenAI 未出现差异。即使短名单不变，最终 ID 仍可变化，因为舍入后的主码分数会进入精排（第3.5.2节）。

7.3.2 打包访问与协作残差评分

按字打包访问在六个数据集、四种 probe 数和 $g_{\mathrm{LUT}} \in \{1, 2\}$ 的 48 对匹配比较中，提升吞吐 1.012–1.658×，召回差最多为 10^{-4}。两种布局均以子空间为主序；改动结合了按字读取与解包（图 7.5）。

精排对照分离了宽载入与协作。L0 是标量参考，L1 只改变载入路径，L16 为每个候选分配 16 lanes。同一轮中轮换变体顺序，共五轮，每轮每变体搜索三次。十二个普通配置的 L16 端到端提升为 1.015–1.789×；独立的高预算 arXiv 对照在 $\alpha = 200$ 达到 2.383×（表 7.5）。

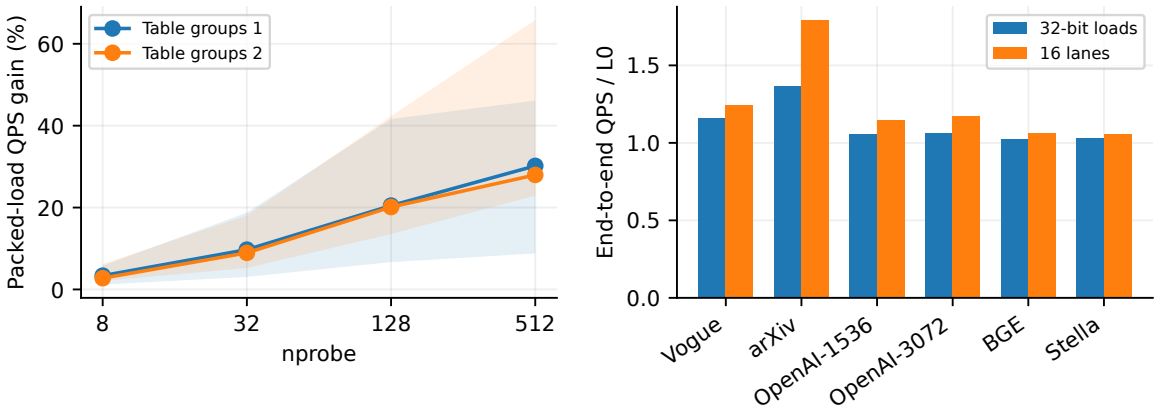


图 7.5. 打包与精排对照。左：各表分组 / probe 数下，六个数据集的打包载入 QPS 收益中位数及最小—最大阴影范围。右：128 probes、各数据集普通预算下，仅改变载入和 16-lane 的实测端到端收益。两个面板使用不同实验家族。

表 7.5. 128 probes 下的精排。比值为相对 L0 的成对端到端收益；末列除外，它是剖析得到的精排内核收益。内核与应用收益不能互换。

数据集	α	$L1/L0$ QPS	$L16/L0$ QPS	内核加速
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

简单的阶段占比模型能接近预测端到端收益。仅改变载入的 L1 保持 L0 的 bits，在 arXiv 上获得 1.368×。采用 lane 协作、$\alpha = 100$ 时，arXiv 精排从 4,277.5 降至 716.8 μs，内核提升 5.97×，整体却为 1.789×。OpenAI-3072 的内核提升为 8.81×，整体为 1.171×。对于基线 wall 时间占比 f 和精排加速比 s，

$$\frac{T'}{T} = (1 - f) + \frac{f}{s}, \qquad \frac{T}{T'} = \frac{1}{1 - f + f/s}.$$

arXiv 采用 $f = 0.529$，预测 1.787×，实测 1.789×；OpenAI-3072 采用 $f = 0.168$，预测 1.175×，实测 1.171×。其他阶段限制了收益，且分阶段剖析独立进行，因此这是解释模型。Lane 重组改变 FP32 加法顺序，相对距离误差最高为 $\approx 1.6 \times 10^{-6}$，偶尔改变排序；其结果是速度提升，而非普遍逐位相同。

7.4 选择正确性与性能

7.4.1 计算分数选择的严格 CPU 参考

全部 55,296 个分组查询实例的主码顺序与 CPU 参考精确一致。单 block 始终满足距离顺序，56 个集合差异均限于截断并列；全部 69,120 个实例对已保留精排分数的最终选择均精确（表 7.6）。

严格 CPU 检查对 GPU 生成的分数按 $(D_{\mathrm{P}}, \mathrm{position})$ 排序。导出器遵循扫描布局和累加顺序，有效返回候选的导出分数逐位匹配。源码比较发现相对表扫描版本只增加导出。270 个配置覆盖 Vogue、arXiv 和 OpenAI-3072，probes 为 1、8、128，$\alpha \in \{4, 100, 200\}$、$g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ 以及两种阈值设置。

表 7.6. 严格选择检查 CPU 参考检查。集合与顺序相等使用完整主码键。有效计数检查 $\min(c, n_c)$ 个保留候选；最终相等按 $(D_{\mathrm{H}}, \mathrm{ID})$ 对保留的计算精排分数排序，包含其 FP32 bits。计数为查询实例，同一查询跨配置复用。

路径	查询数	集合 / 顺序精确	仅并列	有效计数	最终精确
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

全部 19,120 个候选不足案例的有效候选数正确。这些结果检验了 引理 5.1与命题 5.1与5.2 在记录案例中的表现。最终选择相等针对已保留的计算分数，而非原始空间精确近邻；sanitizer 覆盖限于其记录子集（第3.5.2节）。

7.4.2 共享阈值保持选择目标

开启共享阈值后，全部 48 个配置的已查输出保持一致，吞吐比为 0.994–1.562×。收益来自得到有效阈值后，选择与归并维护减少。arXiv 在 $\alpha = 200$、batch 为 1,000、八组时，归并从 2,629 降至 471 μs，扫描从 7,968 降至 5,513 μs。部分配置略慢，说明机制开销仍可能超过省下的工作。

7.4.3 Probe 选择与输出一致性

Radix 选择在全部 1,792 个合成案例中返回相同的计算分数 probes。arXiv、Vogue 和 OpenAI-3072 每个数据集的全部 1,000 条查询中，probe ID、主码分数与主码位置也都匹配参考。改动在已有质心分数中做选择，不改变分数算术或 OpenAI 的精度模式（第7.2.3节）。

表 7.7. radix 选择相对早期 probe 选择器的端到端吞吐比。每个工作负载—batch 比较中，每个二进制有十轮计时，交替运行收集。> 1 的比值有利于 radix 选择。

数据集	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

端到端提升为 1.034–1.167×（表 7.7）。batch 为 8 时，剖析得到 probe 阶段在 arXiv 上从 56.5 降至 22.6 μs，Vogue 从 52.7 降至 14.7 μs，OpenAI-3072 从 77.8 降至 16.7 μs。这些阶段节省在大 batch 总时间中所占比例更小。尚未解决的 Vogue 跨进程差异独立讨论于 第7.8节。

7.5 留出预算质量、吞吐与回本

全池校准在十二个已测配置中均满足留出质量目标；128 查询校准更便宜，却可能频繁失效。对照采用固定 $\alpha = 100$ 与 $\alpha = 64$；规则定义见 第6章。

7.5.1 无标签选择与带标签诊断性 oracle

六个数据集均测试 128 和 512 probes。全池校准使用全部 500 条校准查询；采样校准按第3.2节 的不相交查询池，对每种样本规模取 64 个子集。枚举在全部已测预算上检查同一规则；诊断性 oracle 用留出标签识别满足要求的预算（第6.4节与表 6.2）。全部规则变体使用式 (6.4)，取 $k = 10$ 和 $\epsilon = 1$；标签用于评估所选预算，而非指导校准。

7.5.2 全池校准作为质量优先工作点

使用全部 500 条校准查询时，最大观测留出损失为 0.0006，十二个配置均低于 10^{-3} 目标。吞吐为带标签 oracle 的 0.908–1.000，校准耗时 9.5–60.3 ms（表 7.8）。相对固定 100 的几何平均比为全部配置 1.111×，或固定 100 也满足质量目标的十个配置 1.193×。相对固定

64，校准在六个配置更快、两个持平、四个更慢。主要收益是已观察到的十二个质量目标覆盖，而不是对未来查询分布的保证。

表 7.8. 全池校准（$S = 500$）。损失相对最佳实测留出召回率；速率为所选固定臂的 kQPS。“Cal.” 是每单元一次全池校准调用，不是重复查询流计时。留出查询为 500 个，Stella 为 499 个。

数据集	Probe	α_{full}	损失	Q_{full}	相对 100	相对 oracle	校准 ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

7.5.3 低成本采样及其质量风险

典型所选预算看似适合，个别样本仍可能不达质量目标。$S = 128$ 时，预算中位数在十二个配置中的十个匹配带标签 oracle；差异出现在 arXiv 的 128 probes 和 OpenAI-3072 的 512 probes。不同工作负载的所选预算跨越 4–200（表 7.9）。

表 7.9. $S = 128$ 下的留出质量。规则 α 为 64 次抽样中位数；oracle α 使用留出标签。$\overline{R}_{\text{rule}}$ 为重建的平均召回。“Losses” 计数相对最佳已测留出召回损失 $> 10^{-3}$ 的抽样次数；L_{95} 为日志中的较低次序 95 分位损失估计。R_{100} 为固定预算召回。

数据集	Probe	α: 规则/oracle	R_{100}	$\overline{R}_{\text{rule}}$	越阈数	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

arXiv 在 128 probes 下有 64 次中的 40 次超标，在 512 probes 下为 64 次中的 32 次，尽管平均损失为 0.0009。其固定 100 损失分别为 0.0012 和 0.0016，因此优于继承预算并不直接代表达标。BGE 在 512 probes 有 24 次失败，$L_{95} = 0.0014$；Stella 的 L_{95} 达到 0.0030，而 OpenAI-3072 没有记录到失败（表 7.9）。单次最大损失在 arXiv 为 0.0050、BGE 为 0.0046，均出现在 512 probes。

7.5.4 估计吞吐与校准经济性

采样经常选到更快的预算，但这些速度估计必须与前述质量结果一起理解。采用 第3.5.2节 的所选预算平均估计，$S = 128$ 在十个配置上达到固定 100 的 1.028–1.691× QPS。arXiv 比值为 0.908 和 0.917，平均召回更高，却没有吞吐回本。对固定 64，采样赢得十二个配置中的

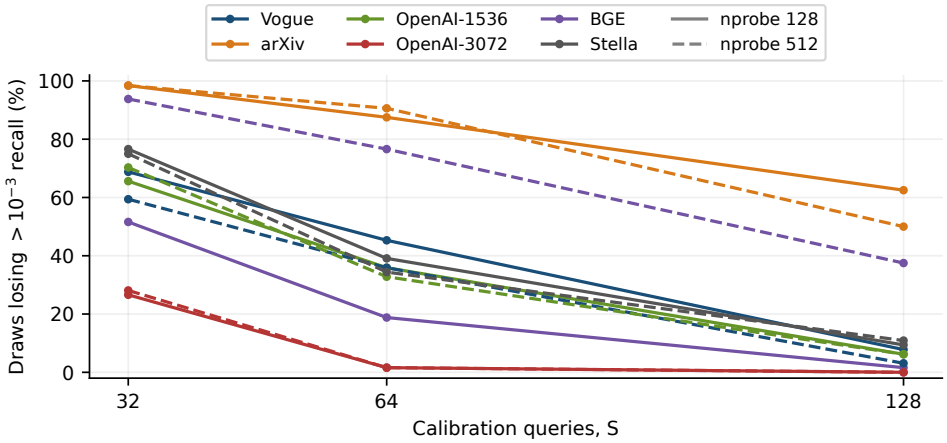


图 7.6. 不同样本量下的留出风险。纵轴为相对最佳已测预算，留出召回损失超过 10^{-3} 的校准抽样百分比。因为 式 (6.4) 保持阈值为一，增加 S 也会收紧允许的槽位不一致比例。

九个，几何平均为 1.054×，范围为 0.821–1.367×；OpenAI 的范围为 1.052–1.367×。两个固定对照都未达到 arXiv 质量目标，因此完整排序并非统一匹配质量。

表 7.10. $S = 128$ 下的校准经济性。速率单位为 kQPS；规则速率是平均所选臂 QPS。比值分别比较固定 100 和 64。“Batches” 用 500 查询、Stella 为 499，给出相对固定 100 的回本次数；“Never” 表示无吞吐回本。质量列单独标明固定 100 是否满足留出损失目标。

数据集	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	相对 100	相对 64	校准 ms	批次数	固定 100 质量
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	达标
arXiv	128	207.9	188.8	0.908	0.821	4.2	不回本	未达标
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	达标
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	达标
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	达标
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	达标
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	达标
arXiv	512	83.1	76.2	0.917	0.870	9.7	不回本	未达标
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	达标
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	达标
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	达标
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	达标

采样校准耗时 3.9–21.3 ms。模型给出的最大正向回本时间，对固定 100 为 0.54 s，对固定 64 为 2.72 s；全池校准则分别为 1.52 和 2.25 s（第6.5节、式 (6.15) 与表 7.10）。后面的直接复用测试同时测总成本与达到的质量。

7.5.5 完整决策轨迹与有限池风险

回放已记录校准选择确认了有限池计算：全部 2,304 次采样决策和十二次全池决策均与枚举一致。其已记录不一致计数曲线均单调，逐查询计数求和复现了全部 18,432 个已测网格计数（式 (6.9)）。这验证的是已观测轨迹，不代表所有距离并列子集或查询排列都满足嵌套（第6.3节）。

$S = 32$ 时单次最大损失达到 0.0294，全池校准则最多为 0.0006。

arXiv 在 128 probes 下，超过损失目标的预测概率与观测概率，$S = 32$ 时为 0.965 对 0.984，$S = 64$ 时为 0.875 对 0.875，$S = 128$ 时为 0.620 对 0.625（图 7.7）。36 个差距中，最大者出现在 BGE 的 512 probes：预测 0.519、观测 0.375，相差 0.144。这些预测针对固定校准池内的样本，结合各所选预算的已测留出质量；校准本身不使用标签。

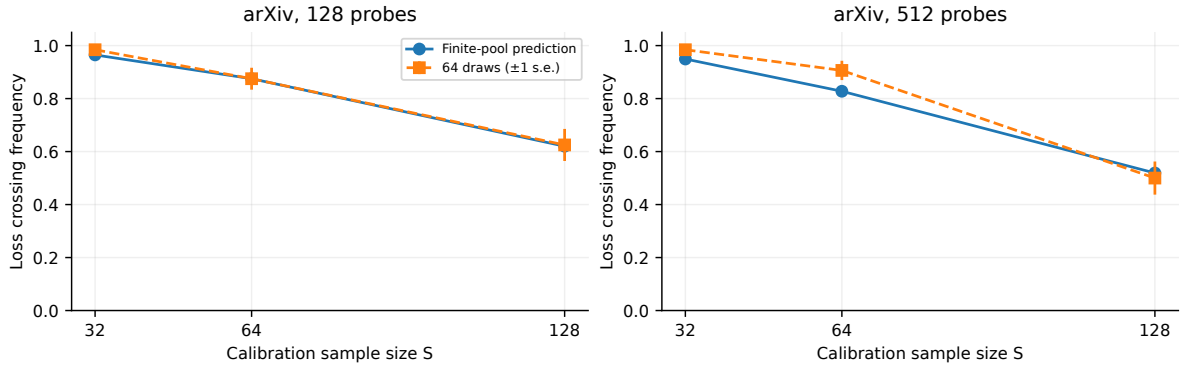


图 7.7. 有限池预测与观测到的 arXiv 损失越阈。误差条为固定池条件下 64 个独立抽样子集的一倍估计标准误，不表示未来工作负载的不确定性。

十次交替计时重复显示小预算平台，而非每次预算变小都会更快的一般规律。

7.5.6 直接测量校准加复用成本

四个配置比较固定预算 64、100、128 查询校准及全池校准，轮换顺序共五轮，每轮 500 个复用 batch。每个 batch 都重复相同的留出查询池，测量的是稳定工作负载上的摊销，而非新查询到达或漂移时的表现。计时包含样本收集、校准、预算安装及首次图设置，不计构建、预热或扫描调度器校准。

表 7.11. 500 个复用 batches 的直接总毫秒数，取五轮平均，抽样与 FULL 策略包含预算校准。每 batch 有 500 查询。质量与计时结果一同讨论。

数据集 / probes	固定 100	固定 64	规则 $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

两个校准策略在 OpenAI-3072 均选择预算 4，召回率为 0.9252，与固定 100 相同。OpenAI-1536 全池校准与固定 100 为 0.9280，采样为 0.9276。BGE 在 512 probes 下，采样为 0.9658、固定 100 为 0.9670、全池为 0.9664。arXiv 的全池策略选择较慢的预算 200 以满足质量目标，两个固定对照则未达标。

对 128 查询校准，两个 OpenAI 数据集的平均累计成本均在第四个 batch 首次低于固定 100。对固定 64，OpenAI-1536 在第六个 batch 越过，OpenAI-3072 在第九个。全池校准对 OpenAI-1536 分别需要九个和十四个 batch，对 OpenAI-3072 为十一个和二十七个（图 7.8）。这些实测交点补充了稳态速率模型；召回率不同的策略不能只按时间排序。

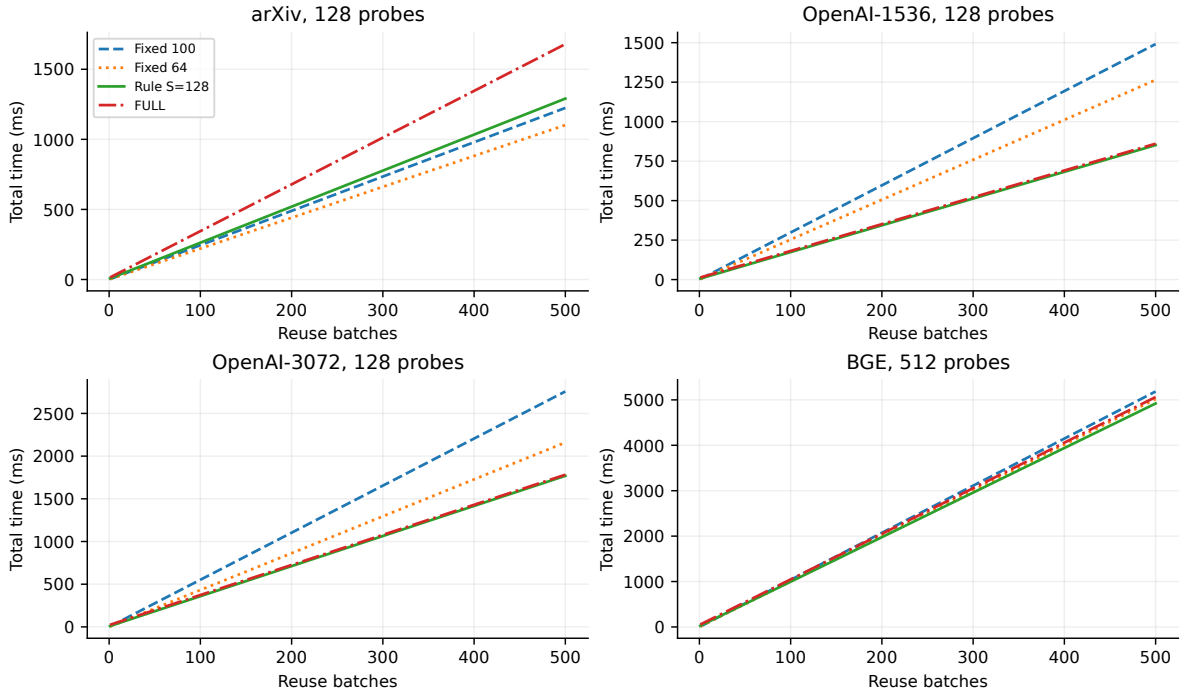


图 7.8. 直接测得的平均累计时间，包含预算校准和首 batch 准备。每条曲线重放一个固定留出查询池。不同策略可能有不同召回率；仅靠时间不能形成质量匹配排名。数据集 / probe 设置与 表 7.11 相同。

7.6 Batch 大小、调度与工作负载不均衡

在已查配置的 250 对变体 / 配置和 234,320 次查询比较中，调度没有引入非并列主码短名单差异或最终 ID 集合差异。并列主码的保留成员或输出顺序可以不同。性能依赖 batch 大小：相对采用 lane 精排的单 block 基线，集成调度器在 batch 为 8 时平均提升 2.727×，batch 为 1,000 时为 1.015×（表 7.12）。这些平均值不表示每个配置都改善。

7.6.1 并行度与重复选择工作

固定八组扫描在 batch 为 8 时获得 2.06–2.76×。arXiv 扫描从 849 降至 216 μs，另有 16.7 μs 归并。batch 为 1,000、$\alpha = 200$ 时，采用独立阈值的分组却使扫描从 3,530 增至 7,652 μs，并增加 2,624 μs 归并，仅达到单 block 的 0.46× QPS。共享阈值减少维护工作，但不能消除过度分组的代价（图 7.9）。

7.6.2 固定规则在更多配置上的评估

规则常数先在四个配置上确定，再测试其余配置（第5.5节）。

表 7.12. 相对单 block、十六 lane 基线的集成执行收益。条目为五轮逐单元成对加速比中位数的几何平均。“Held out”排除确定规则的四个配置，表示留出配置，而非预算实验的留出查询划分。

Batch	单元数	$g_s = 2$	$g_s = 4$	$g_s = 8$	调度	留出配置
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

平均值覆盖十三个小 batch 配置和十一个大 batch 配置，Stella 采用 999 条查询。排除用于制定规则的四个配置后，分别剩九个和七个（表 7.12）。batch 为 1,000 时，固定八组平

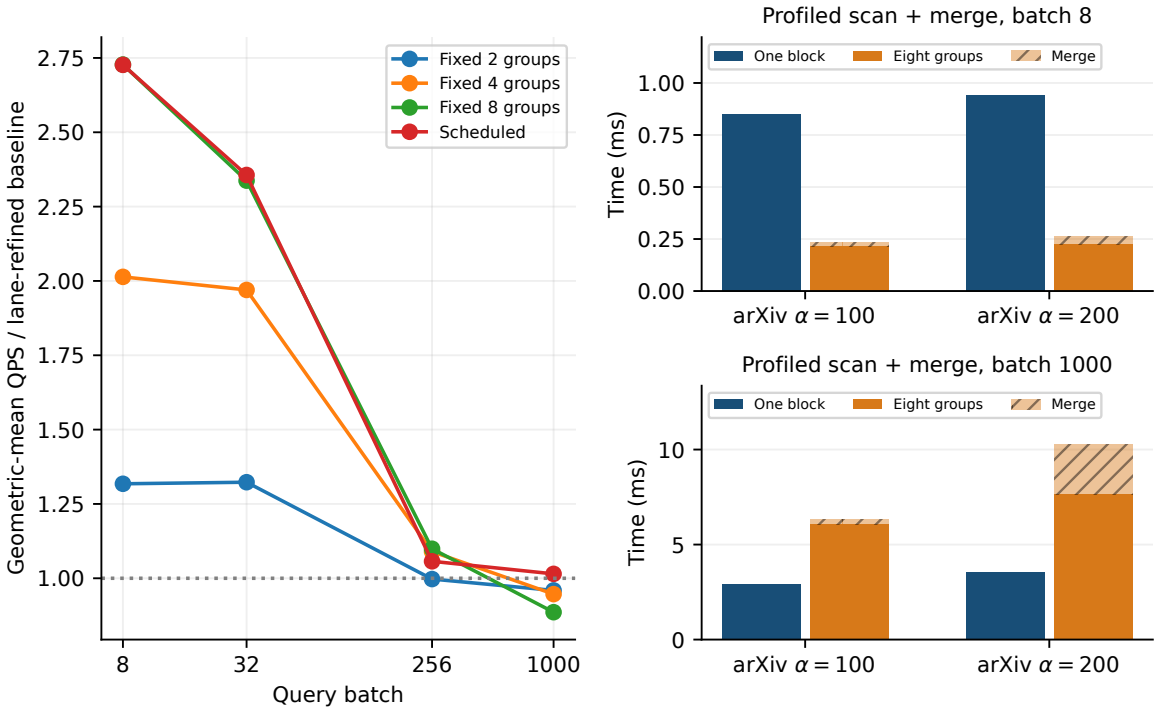


图 7.9. 执行区间。左：固定共享阈值组数及调度执行，相对 lane 精排单 block 基线的速率；小 batch 有 13 单元，batch 1,000 有 11。右：arXiv 预算 100、200 在 batch 8、1,000 下的成对扫描 / 归并剖析。斜纹表示归并。右侧来自独立阈值扫描实验，解释一种成本，不是调度器的完整加速。

均为基线的 0.886×。与最佳固定选择相比，调度器耗时最多为其 1.217 倍，发生在 BGE 的 batch 256。

Vogue 说明候选数变化为什么是不完整信号。其 CV 为 0.338–0.622，即使 batch 为 1,000 也选择四组。调度 / 单 block 比值为 0.895–1.175，低 probe 配置会因总工作太少、无法补偿分组而变慢。同一选择带来了召回率 0.97 的优势，但规则没有显式估计扫描总量或归并成本（第7.2节）。

7.7 索引构建、常驻显存与已测规模

JHQ-GPU 能在单张 GPU 上构建和搜索最大已测数据集：Stella 的 17.78 百万个向量在搜索时占用 20.44 GiB。RQ1 分别通过构建、常驻显存和已完成的搜索测量评估；分配比较也回答 RQ4。

7.7.1 全新索引与输入加载成本

这里的冷构建指索引冷态：不复用训练状态缓存。正常构建测量使用已在操作系统页面缓存中的输入文件，计时 train 和 add 调用，不计文件加载或完整进程启动。三次重复中，Vogue 训练耗时 298–303 ms，总计约 ≈ 0.56 s；Stella 为 10.54–10.65 s（表 7.13）。

重复构建比较见 图 7.10，计时边界定义于 第3.3节与第3.5.2节。

表 7.13. 冷 JHQ 构建与单独测量的常驻显存。Train / add 为三次补充正常构建的组件中位数；Total 为成对总和中位数。显存来自查表扫描实现的两组、128 probes、$\alpha = 100$ 运行，包含上下文和工作区。“Residual” 是残差编码模型大小占实测使用总量的比例，不是每查询读取比例。

数据集	训练 s	Add s	合计 s	使用 GiB	原始 GiB	残差占比
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

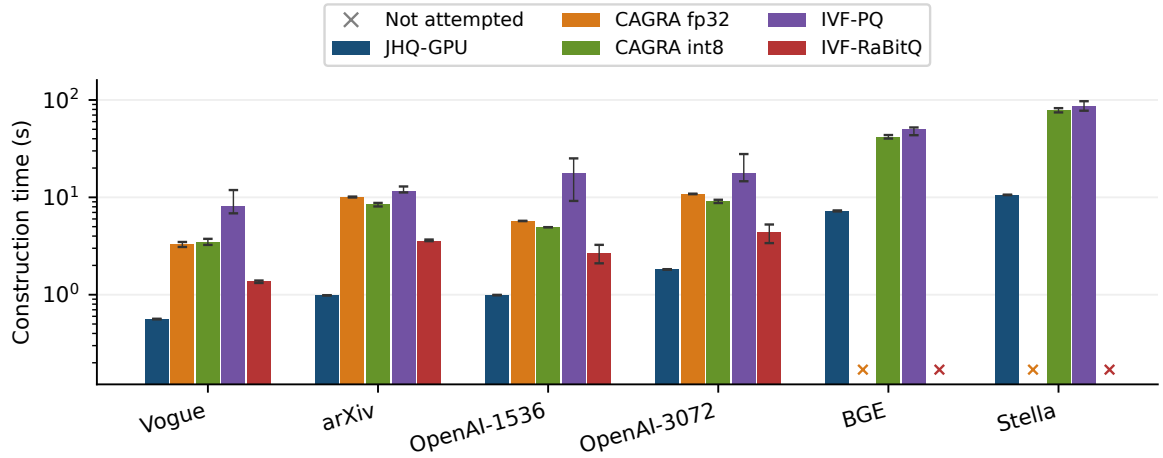


图 7.10. 记录的构建成本，纳入 CAGRA int8。JHQ 为输入页已热时三次补充正常冷构建的中位数及范围；其他柱为成功测量索引配置的中位数，误差线为最小—最大值。Int8 包含数据集量化；RaBitQ 不含所需序列化往返。缺失大数据集柱表示未尝试，不是实测构建失败。纵轴为对数。

Stella 的两个独立 CAGRA int8 配置分别耗时 74.55 和 82.45 s，含量化；IVF-PQ 三个配置的中位数为 86.59 s，而 JHQ 为 10.54–10.65 s。这些是构建比较，并非质量和显存相同的搜索工作点。

五次重复训练中，arXiv 召回率变化最多为 0.0012。输入页已缓存时，Stella 从启动到可搜索需 18.2–18.6 s，本会话首次读取时需 129.9 s。后两者包含输入加载与启动，因此高于 train 加 add 的时间。每 20 ms 采样的显存峰值，BGE 为 26.08 GiB，Stella 为 22.58 GiB。BGE 的 12 GiB 分配缓冲区在 23.05 GiB 的 gather 载荷之前释放，不能相加为同一瞬时峰值（式 (4.13)）。

7.7.2 常驻分配与压缩的作用

压缩使大规模常驻索引成为可能，但残差仍占主要存储。图 7.11 将 第4.4节 的组件模型与实测总量结合，保留未归因余量。

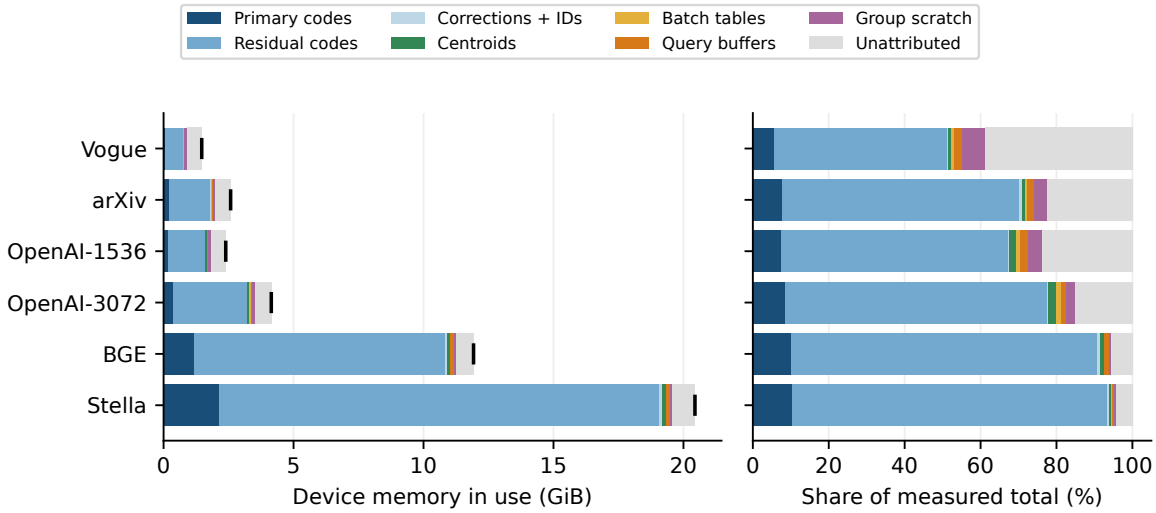


图 7.11. 两组表、128 probes、$\alpha = 100$ 下的常驻显存分解。组件大小为模型值，标记为实测使用总量，斜纹为未归属余量。总量包含上下文和工作区。前述构建测量给出采样峰值；这些常驻标记不是峰值。

残差占 BGE 的 11.93 GiB 的 80.7%，占 Stella 的 20.44 GiB 的 82.9%，总量包括上下文和工作区。两者的原始 FP32 向量分别需要 38.50 和 67.81 GiB。降低 α 节省残差读取，不减少已存残差字节。

跨系统分配采用不同边界（表 7.14）。从 JHQ 进程总量减去估计为 507.2 MiB 的 CUDA 上下文后，净进程分配约为 CAGRA fp32 索引分配的 0.29–0.39。JHQ 仍包含查询工作区，CAGRA 报告构建时的索引增量。CAGRA int8 的分配与 JHQ 同量级。这些是参数扫描范围，并非相同显存的召回工作点，也不是相同峰值预算；组件模型见 表 4.1。

表 7.14. 主比较分配范围。JHQ 为进程总量减去估计的 507.2 MiB 上下文，包含查询工作区；两个 CAGRA 列是构建时测得的索引分配。范围覆盖参数扫描。

数据集	JHQ 净值进程 GiB	CAGRA fp32 索引 GiB	CAGRA int8 索引 GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

7.7.3 大数据集可行性与竞争区间

JHQ 在两个大数据集上的最高实测召回率均高于成功运行的压缩基线，而 CAGRA int8 在部分低召回区间领先（图 7.12）。BGE 有 10,091,524 个向量，Stella 有 17,776,615 个，均为 1,024 维，采用 32,768 个 IVF 列表。Stella 使用从语料留出的查询，评估范围见 第3.2节。

在预计存储超过设备容量时，没有尝试完整精度 CAGRA、RaBitQ 和 FP32 暴力搜索。缺失点是预先排除，不是实测显存不足或召回失败。

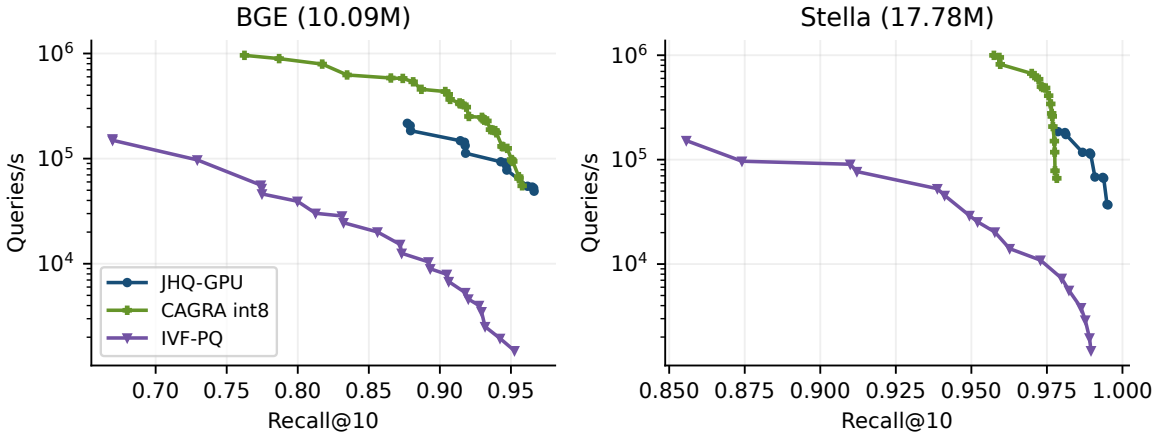


图 7.12. 大数据集主机到主机前沿，纳入成功的隔离 CAGRA int8 重跑。BGE / Stella 使用 1,000/999 查询。端点标记已测网格覆盖。

CAGRA int8 可以容纳 Stella。BGE 在召回率 0.88–0.95 时，它快 1.35–2.96×，而在 int8 的最高召回率 0.9579 处，JHQ 快 1.086×。BGE 和 Stella 的最高召回率，int8 分别为 0.9579、0.9783，JHQ 为 0.9663、0.9952，IVF-PQ 为 0.9525、0.9896。

Stella 上，JHQ 在召回率 0.978679 达到 185.4 kQPS，int8 在 0.978278 达到 66.3 kQPS。2.80× 比值比较接近的实测点，并非相同召回率；int8 没有更高区间的已测覆盖。

7.8 数值检查与召回损失分析

7.8.1 质量分辨率与重复执行

降低慢 RaBitQ 调用影响后，召回率 0.97 的排序仍然保持，但接近持平的结果对数值和测量分辨率敏感。一个真实近邻命中使 Recall@10 改变

$$\Delta r = \frac{1}{nk} = \begin{cases} 10^{-4}, & n = 1000, \\ 2 \times 10^{-4}, & n = 500. \end{cases}$$

Stella 的 $n = 499$ 步长略大；这些步长是测量分辨率，不是不确定区间。

召回率 0.97 附近，应用插值容差前，局部斜率对应每 10^{-3} 召回变化 1.01–3.17% QPS。五次构建的召回范围，Vogue 为 0.0008，arXiv 为 0.0012，OpenAI-1536 为 0.0004，OpenAI-3072 为 0.0002。这些说明局部敏感性，不能界定完整重训前沿。缓存模型下的 FP32 比较在 OpenAI 上仍有利（第7.2.3节）；arXiv 和 LUT 接近持平的结果仍不确定。

RaBitQ 的五轮 QPS 中位范围在 Vogue 为 3.79%，arXiv 为 3.34%，OpenAI-1536 为 2.87%，OpenAI-3072 为 5.25%。这是已观察到的波动范围，不是置信区间。为检验偶发 30–60 ms 调用的影响，将 1,000 查询速率改按 $10^6/\,\text{median(call ms)}$ 计算，而非均值调用时间。召回率 0.97 的 JHQ / RaBitQ 比值依次变为 1.063、0.995、1.118、1.100：三个有利方向与 arXiv 接近持平的结论保持。

Vogue 在身份检查与计时运行之间，召回率从 0.9642 变为 0.9644，基线自身的运行也出现此差异。另一次跨二进制身份检查中，1,000 条查询的 probe 与主码转储逐字节一致，最终召回率却不同。这些现象仍未解决；同进程内的选择检查不能解释跨进程变化（第3.5.1节）。

7.8.2 重建分数与召回损失

重建 pilot 在已查案例中量化了较小的剩余预算相关差距，但路由与表示仍是合并局限。七个工作负载与预算配置各取 128 条查询，对全部已探测候选做 FP64 评分，相对 GPU 所选预算输出，召回提高 0–0.002344。arXiv 在 128 probes 下，$\alpha = 200$ 在全部 128 条查询中

返回与全 probe 重建评分相同的答案集合。预算为 100 时，召回为 0.9734375，全 probe 评分为 0.9750，差距为 0.0015625，同时存在合计 0.0250 的路由 / 重建损失（第6.2节）。

这个例子没有分离路由与重建，其他配置的比较还混入 FP64 / GPU 算术差异。完整归因需要固定查询和候选集合，逐阶段测量真实近邻覆盖。表 5.1 的阶段检查验证的是计算分数上的选择正确性，而非原始空间精确最近邻。

第 8 章讨论与未来工作

本章解释压缩和并行执行何时有益、为什么放弃若干替代设计，以及哪些实验可能继续改进系统。

8.1 设计启示

8.1.1 索引常驻与构建工作区

压缩决定索引能否常驻 GPU，但构建还有独立的容量约束。BGE 搜索时占用 11.93 GiB，构建采样峰值却达到 26.08 GiB。这一峰值来自分配缓冲区，发生在 gather 缓冲区分配之前。两遍构建限制了 gather 存储，并未消除其他临时分配。因此，可行性需要同时依据常驻字节模型和逐阶段分配模型判断（第4.5节与第7.7节）。

8.1.2 码本结构与查表开销

最小的表并非最快。四路和八路拆分每个子空间存 16 项，少于两路拆分的 32 项，却需要更多查表和加法。两张表在 24 个配置中的 22 个最快（第7.3节）。类似地，完全去掉查找表虽然节省共享内存，在 表 8.1 汇总的早期测试中仍然变慢。因此，应当比较实际 GPU 评分成本下的重建质量。

8.1.3 并行度必须补偿选择与归并开销

小 batch 中独立查询太少，难以填满 GPU。把每个查询分到多个 block 能增加工作量，但会重复暂存查找表和局部选择，还要增加归并。batch 为 8 时，八组获得 2.06–2.76× 加速；batch 为 1,000 时，采用独立阈值的分组在已剖析的高预算配置中，吞吐降至单 block 的 0.46×（第7.6节）。这不意味着大 batch 总应使用单 block：分组也能处理不均衡负载，并带来了 Vogue 在召回率 0.97 处的领先。其低 probe 配置上的吞吐比 0.895 则体现了规则的局限。候选数的离散程度并不等于总量；更好的调度应同时估算扫描、选择和归并成本。

8.1.4 选择正确性与检索质量

按已计算分数精确选择，与找到原始向量的最近邻，是两个不同要求。分组选择器的证明和 CPU 排序检查说明，并行拆分仍能保留规定的主码短名单（第5.4节与第7.4.1节）。这不会消除 IVF 路由损失或量化误差。分开验证这些层次，才能区分实现缺陷与表示、精排预算的局限。

8.1.5 精排速度与预算收益

较小预算能够省下多少时间，取决于搜索的其他部分。令 $T_0 > 0$ 为固定的非精排时间，$F_h \geq F_\ell > 0$ 为高、低预算的精排时间，$\eta \in (0, 1]$ 为使两个精排内核一起加速的共同倍率。低预算加速比及其导数为

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$

$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

当 $F_h > F_\ell$ 时，降低 η 会降低削减预算的相对收益，而降低 T_0 会增大这一收益。这个模型解释了为何不能把内核优化和预算削减当作独立、可直接相乘的收益。它假设两个预算下精排加速相同，并且非精排工作固定。实际扫描维护成本可能随预算改变，因此是否采用仍应依据总时间和质量的实测（第7.5节与第7.5.6节）。

8.2 替代评分与编码实验

8.2.1 Mailman 式共享求和评分

JHQ 的有限符号模式提示，可以参照 Mailman 思想在候选之间复用部分和 [20]。四位半码只有 16 种模式，因此许多候选可以共享中间计算。测试在 Vogue、arXiv 和 OpenAI-3072 上固定主码及候选流，将分组宽度为 4、6、8 的原子桶和私有直方图，与按字打包的两表评分比较，采用 128 probes 和 8、1,000 两种 batch。

最佳替代方案的吞吐也只有评分基线的 0.187–0.581。剖析显示桶更新集中；私有直方图则增加状态和共享内存更新。当实现复用的成本超过省下的查表工作，算术复用就不足以带来收益。这些是评分器测量，并非完整搜索对比。若 f 为基线中评分占用的时间比例，s 为替代评分器的相对速度，其余部分不变的模型给出

$$\frac{T'}{T} = 1 + f\left(\frac{1}{s} - 1\right) > 1 \quad (0 < f \le 1,\ 0 < s < 1).$$

因此，即使这个模型也预测不了整体搜索收益。集成在此停止，其他有限字母表实现仍值得探索。图 8.1 左图展示六组评分器比较。

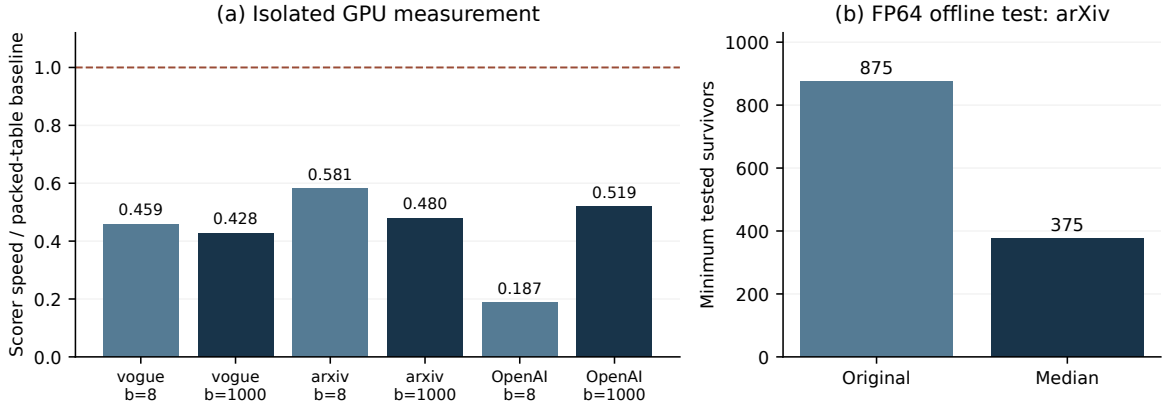


图 8.1. Mailman 评分器吞吐（左）与离线短名单大小（右）。

8.2.2 面向排序的编码实验

最小化重建误差未必保持近邻顺序。FP64 离线回放采用 400 条校准、200 条验证和 400 条测试查询，检查加权分数、中位数及列表相关阈值、学习损失和线性最小均方误差解码。arXiv 上，中位数阈值将达到召回率 0.97775 所需的最小已测短名单从 875 个候选降至 375 个；另外两个工作负载未达各自目标。这些是 图 8.1 右图的离线计数。

独立的 GPU 后续实验同时改变主码表示并重训残差码。在它自己的 arXiv 目标 0.9766 下，batch 为 8 时吞吐提升 1.40×，batch 为 1,000 时提升 1.21×。Vogue 的最高召回率却只有 0.9630，低于目标 0.9644；OpenAI-3072 接近持平。离线候选减少与 GPU 加速使用不同的查询、训练态和目标，不能据此得到只改变阈值的加速比。主码改变后，残差需要重建的对象也会改变。

8.2.3 移除共同均值

全局数据均值可能占用符号码的分辨能力。旋转保持均值的范数：对于正交的 Π，

$$\bar{y} = \frac{1}{N}\sum_i \Pi x_i = \Pi \bar{x}, \qquad \|\bar{y}\|^2 = \bar{x}^\top \Pi^\top \Pi \bar{x} = \|\bar{x}\|^2.$$

在实数算术中，从查询和数据库向量中减去相同均值 μ，保持距离 $\|(q-\mu)-(x-\mu)\|^2 = \|q-x\|^2$，却会改变待量化的问题。

后续实验将存档的原始索引与在中心化数据上新训的索引比较，在同一会话交替测量。arXiv 的均值范数为 0.837，采用 128 probes 时，全池校准从 $\alpha = 200$ 改选 $\alpha = 64$，召回率 0.97 处的前沿吞吐提升 1.266×。512 probes 时所选预算变为 100，并非 64。这一变化并非总有利：Vogue 在 512 probes 下的高预算召回率从 0.9918 降至 0.9896；OpenAI-1536 的所选预算从 8 升到 16，尽管召回率 0.97 处吞吐比 0.995，几乎不变。arXiv 的结果支持将去均值作为有用的表示改动。重训同时改变质心和残差，因此还需对两种数据做同样的重复重训，才能分离其贡献。这些测试扩展的是分析，并未并入与其他系统的主比较。

8.2.4 安全下界仍可能剪枝不足

另一项 FP64 pilot 仅根据主码，为到重建向量的平方距离构造下界。它按符号模式统计覆盖全部已索引向量的残差坐标范围。将包围盒加上对应的主码重建后，以查询到它的距离作为下界。在七个搜索配置、7,280 万次候选检查中未观察到违反；但下界与距离之比的中位数只有 0.13–0.24。即使用理想的最终阈值，几乎所有已探测候选仍要精排，通常远多于现有短名单。因此 pilot 在完整 GPU 实现之前停止。安全性还不够：剪枝必须足够有选择性，才能补偿下界计算和存活残差读取的成本。GPU 版本还需采用考虑舍入安全的包络。

8.2.5 其他替代方案与停止决定

表 8.1 汇总代表性的早期替代方案。它们各用自己的版本和对照，而非最终跨系统协议。GPU 计时与离线成本估算分别标明；后者不是额外的系统前沿测量。

表 8.1. 替代设计与停止决定。

替代方案	预期节省	证据与决定
更大的选择缓冲区	减少压缩排序	GPU 测试吞吐最多下降 24%；增加共享内存后，受影响配置可常驻的 block 数减少
扫描与精排融合	减少启动和中间传输	GPU 测试最多下降 15%；省下的传输较少，却增加了合并内核的资源需求
部分分数提前退出	跳过后续子空间	GPU 变化为 −2.8 至 +4.2%；warp 级检查没有稳定收益
免表符号评分	去掉查找表存储	尽管共享内存更少，GPU 吞吐仍下降 30–52%
渐进读取主码	只为存活候选读取完整编码	离线回放和访存估算显示第二阶段散乱读取的成本；未计时新的 GPU 搜索内核
列表内优先精排	避免全局主码归并	离线配额策略在相同工作量下未超过全局选择；有利的 oracle 分配在线不可得

8.3 内部与外部有效性的威胁

表 8.2 列出主要解释可能失效的原因以及对应对照。测量定义仍见第 3 章；重复构建和精度检查见 第7.8节。

表 8.2. 有效性威胁与剩余检查。

受威胁主张	替代解释	对照与剩余测试
执行收益	分数、并列或索引状态改变造成表观收益	CPU 重排序与表分数检查；覆盖有限，进一步测试需重新生成候选数组
预算质量	校准样本遗漏困难查询，或工作负载改变	独立的带标签测试查询、已记录决策曲线和全池枚举；仍有未测查询子集与未来分布
竞争优势	参数网格、慢调用或接口开销改变排序	两种召回估计器、保留慢调用、int8 与精度检查、小 batch 比较；其他设备未测
容量 / 构建主张	短生命周期分配逃过峰值采样	分配生命周期、重复冷建和 20 ms 显存采样；更短峰值及完整重训练前沿仍未测

第7.2.7节 中较早的 CPU 比较给出了历史实现在相同召回率下的吞吐差异。独立训练、OpenAI 数据集的列表数不等以及 CPU 计时重复较少，限制了这些比值的解释。针对本文评测 GPU 版本、匹配索引和计时条件的比较仍未完成。

六个数据集不代表六种独立用户工作负载。两个 OpenAI 导出共享 DBpedia 语料，Stella 的查询从语料中留出，而非原始查询集（第3.2节）。比较包含传输与主机接口；内存类型及 C++ 与 Python 调用成本的差异可能影响小 batch 的小幅差距，因此不能全部归因于 GPU 搜索内核（第3.3节）。证据覆盖一张 GPU、静态索引和平方 L2 top-10 搜索。可能仍有近重复，并且缺少相关性标签；更新、并发服务、漂移、十亿规模容量和多 GPU 均未评估。

8.4 未来研究方向

8.4.1 考虑均值的表示

中心化结果提示，应研究主码符号何时应编码相对均值的偏差（第8.2.3节）。首先需要对中心化和未中心化数据重复冷建，以区分其效果与训练波动。列表相关的均值也是候选方向，但要求查询评分与残差重建保持一致。测试应比较短名单覆盖率、可达到的最佳精排召回率、所选预算和总吞吐，防止较便宜的短名单掩盖质量上限下降。

8.4.2 校准风险与复用成本

具有统计控制的预算选择，能否省下比获取预算更大的查询时间？在 式 (6.11) 的嵌套条件下，参考近邻的主码排名可代替反复试搜。早期四配置离线回放发现，序贯抽样增加了搜索次数；用已测校准时间拟合的模型估算，即使样本更少，成本仍然更高。这不是完整 GPU 控制器的测量。替代方案应测试批量试验或排名复用，并以直接校准加复用总时间，对比质量匹配的固定预算。风险分析必须覆盖抽样和预算选择程序，发生漂移时应更新校准；相对参考的控制仍需与标签建立联系，才能给出绝对召回保证。

8.4.3 GPU 与主机内存间的残差存储

BGE 和 Stella 的残差占进程总常驻显存的 80.7–82.9%（第7.7节），但只有短名单中的行需要读取。将路由和主码留在 GPU，按块从主机内存取回残差，有望增加容量。难点在于把分散的短名单位置转成高效块传输。在评分与选择固定时，改变设备容量上限，测量额外字节、缓存命中、传输重叠、吞吐和尾延迟。由此才能确定容量与延迟的取舍，而非假定卸载会提高速度。

8.4.4 解释小 batch 的边界

在召回率 0.97 处，分组改善了 JHQ 的小 batch 执行，却没有消除与 CAGRA fp32 的差距（第7.2.6节）。候选工作量、选择开销和接口成本都可能参与其中。匹配的研究应统计两种索引评分的候选数，并区分内核时间、传输和主机调用开销。跨 batch 比较这些成本，才能检验下一步应减少已探测候选、降低选择成本，还是使用不同索引。

8.5 部署建议

已测结果使 JHQ-GPU 成为显存受限、召回率接近 0.97 的批吞吐工作负载的候选（第7.2节与第7.7节）。这不代表到该召回率之前都占优：压缩基线在部分低召回区间获胜，CAGRA fp32 则在 batch 为 8、召回率为 0.97 时获胜。对于稳定工作负载，已测的 500 查询校准可作为初始设置，随后独立核查质量，并与固定预算比较复用总时间。索引或查询分布改变后，需要重新验证。

第 9 章结论

9.1 对四个研究问题的回答

RQ1：GPU 索引构建与容量。 完整 JHQ 索引能够在单张 GPU 上构建和搜索，无须把原始数据库向量保存在设备上。训练和编码均在 GPU 执行，通过有界批处理与两遍构建控制临时存储。Stella 的 17.78 百万个向量若以 FP32 保存，需要 67.81 GiB；JHQ-GPU 的常驻进程占用为 20.44 GiB，包括上下文与工作区。索引冷态构建耗时 10.54–10.65 s，在这张 32 GB 设备上采样到的峰值为 22.58 GiB（第7.7节）。

RQ2：面向表示结构的执行。 JHQ 的编码结构支持小查找表、连续读取和并行候选选择。每张 256 项主码表拆成两张 16 项表后保持实数算术分数，并在 24 个配置中的 22 个成为最快的已测表形式。编码按字打包，使扫描相邻候选的线程读取连续内存；多个线程还可以协作精排一个候选。小 batch 下，将一个查询分到多个 block 能增加有效并行度：batch 为 8 时，调度相对采用协作精排的单 block 基线获得 2.727× 吞吐。分组选择在全部 55,296 个已查查询实例中，与 CPU 对 GPU 分数严格排序的结果一致；单 block 的差异仅出现在截断边界的并列候选上（第7.3节、第7.4.1节与第7.6节）。

RQ3：工作负载级精排校准。 无标签查询能够选出可复用且满足已测质量目标的精排预算，但样本数量会影响结果。使用全部 500 条校准查询时，十二个配置的留出集召回率与已测预算中最高召回率的差距均在 10^{-3} 以内，最大观测损失为 0.0006。小样本可能漏掉需要更多精排的查询：采用 128 条查询时，arXiv 在 128 probes 下有 64 次抽样中的 40 次超出目标，在 512 probes 下为 64 次中的 32 次。采用 128 查询校准时，OpenAI 直接复用测试相对固定预算对照在 4–9 个 batch 内收回开销。arXiv 选出的预算则需要花更多时间来达到质量目标(第7.5节)。

RQ4：跨系统搜索性能。 主比较显示 JHQ-GPU 适合大 batch 吞吐场景。在召回率 0.97、batch 为 1,000 时，它在四个数据集上达到 CAGRA fp32 的 1.32–1.75× 吞吐；在已测网格中，扣除 CUDA 上下文后的净进程显存为 CAGRA 索引分配量的 0.29–0.39。它在 Vogue 和两个 OpenAI 数据集上领先已测方法，在 arXiv 上与 IVF-RaBitQ 接近持平；Vogue 的领先依赖分组扫描（第7.2节）。OpenAI 测量使用 TF32 乘法；复用相同训练模型的 FP32 检查仍领先 CAGRA fp32（第7.2.4节）。这一优势受到 batch 大小限制：batch 为 8、召回率为 0.97 时，CAGRA fp32 在四个数据集上均更快（第7.2.6节）。

9.2 贡献、局限与展望

当执行方式使编码访问成本降低、并行度匹配 batch、精排满足工作负载的质量需求时，压缩才能带来容量和速度收益。

证据覆盖一张 GPU、六个准备后的数据集、平方 L2 top-10 搜索和静态索引。历史 CPU 比较不能量化本文评测版本的加速比（第7.2.7节）。在校准池上与参考结果一致，也仍不能确定未来查询的质量。

由此可以继续研究两个方向。风险受控的校准应考虑工作负载漂移和预计复用次数，并与质量匹配的固定预算比较包含校准开销的总时间。从主机内存取回残差码有望增加容量；测量传输量、传输与计算的重叠以及尾延迟，可判断增加容量的代价是否值得（第8.4节）。

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record. Project AI-use record, 2026. Implementation and experiment records; original prompts and interaction metadata unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages

331–340, 2012. doi: 10.1145/2304576.2304621. URL https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL https://proceedings.mlr.press/v119/guo20h.html.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL https://arxiv.org/abs/1702.08734.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL https://www.cs.yale.edu/publications/techreports/tr1402.pdf.

[21] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320, 2016. URL https://arxiv.org/abs/1603.09320. Version 4, revised August 2018.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html.

PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record. Project AI-use record, 2026. Retained review excerpts; original prompts and interaction metadata unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions. Project AI-use record, 2026. Selected prompts and uses; compiled 29 October 2026. Partial record; historical interaction metadata unavailable.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL `https://proceedings.neurips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html`.

[35] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[36] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[37] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[38] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085. URL `https://ssomesh.github.io/docs/papers/tbd25.pdf`.

[39] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[40] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.